

KUBERNETES (K8S) TASK - 01

1. Setup in your local machine.

Setup Minikube on Windows — Step by Step

Objective: To install and configure Minikube on a local machine and create a single-node Kubernetes cluster for learning and practicing Kubernetes concepts such as Pods, Deployments, Services, Namespaces, ConfigMaps, and other Kubernetes resources.

1. Prerequisites

Since you are using **Windows + Git Bash**, I recommend using:

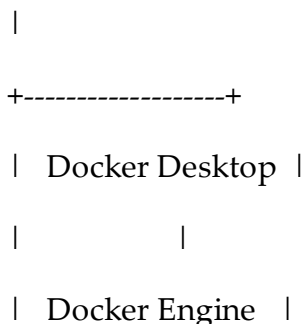
- Windows 10/11
- Docker Desktop
- Minikube
- kubectl
- Internet connection
- At least **4 GB RAM** available
- At least **20 GB free disk space**

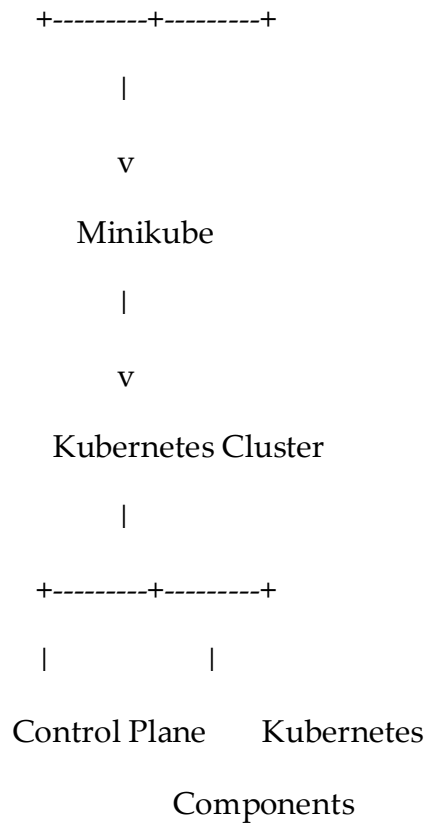
We will use **Docker as the Minikube driver**.

Architecture

After installation, your local setup will look like this:

Windows Laptop/Desktop





Step 1 — Check Your System

Open **PowerShell** or **Git Bash**.

Check Windows version:

`winver`

You should have a reasonably recent Windows 10/11 system.

Also check whether virtualization is enabled.

In PowerShell:

`systeminfo`

Look near the bottom for:

Virtualization Enabled In Firmware: Yes



```
Administrator: Windows PowerShell

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Windows\system32> systeminfo

Host Name:                RAJKUMARI
OS Name:                   Microsoft Windows 11 Home Single Language
OS Version:                10.0.26200 N/A Build 26200
OS Manufacturer:          Microsoft Corporation
OS Configuration:          Standalone Workstation
OS Build Type:              Multiprocessor Free
Registered Owner:          brajkumari7675@gmail.com
Registered Organization:    N/A
Product ID:                 00342-42771-92037-AAOEM
Original Install Date:      08-04-2026, 05:55:22
System Boot Time:           16-09-2026, 20:38:31
System Manufacturer:        LENOVO
System Model:               83ER
System Type:                x64-based PC
Processor(s):                1 Processor(s) Installed.
                             [01]: Intel64 Family 6 Model 154 Stepping 3 GenuineInt
BIOS Version:               LENOVO LTCN44WW, 01-04-2026
Windows Directory:          C:\Windows
System Directory:           C:\Windows\system32
Boot Device:                 \Device\HarddiskVolume1
System Locale:               en-us;English (United States)
Input Locale:                00004009
Time Zone:                   (UTC+05:30) Chennai, Kolkata, Mumbai, New Delhi
Total Physical Memory:       16,077 MB
Available Physical Memory:   6,893 MB
```

Step 2 — Install Docker Desktop

Minikube needs a container runtime. For your local setup, we'll use **Docker**.

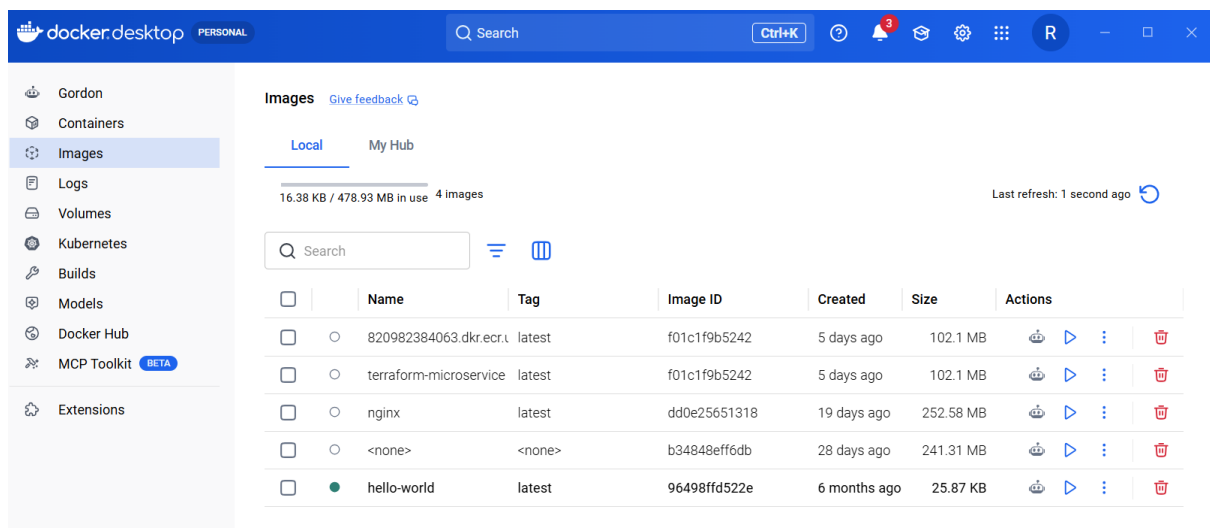
Download Docker Desktop from the official Docker website:

[Docker Desktop for Windows](#)

Install Docker Desktop.

During installation, keep the **WSL 2** option enabled if offered.

After installation, restart your computer if requested.



Step 3 — Start Docker Desktop

Open:

Start Menu



Docker Desktop

Wait until Docker Desktop shows that Docker is running.

Then open Git Bash and run:

```
docker version
```

You should see something similar to:

Client:

Version: ...

Server:

Engine:

Version: ...

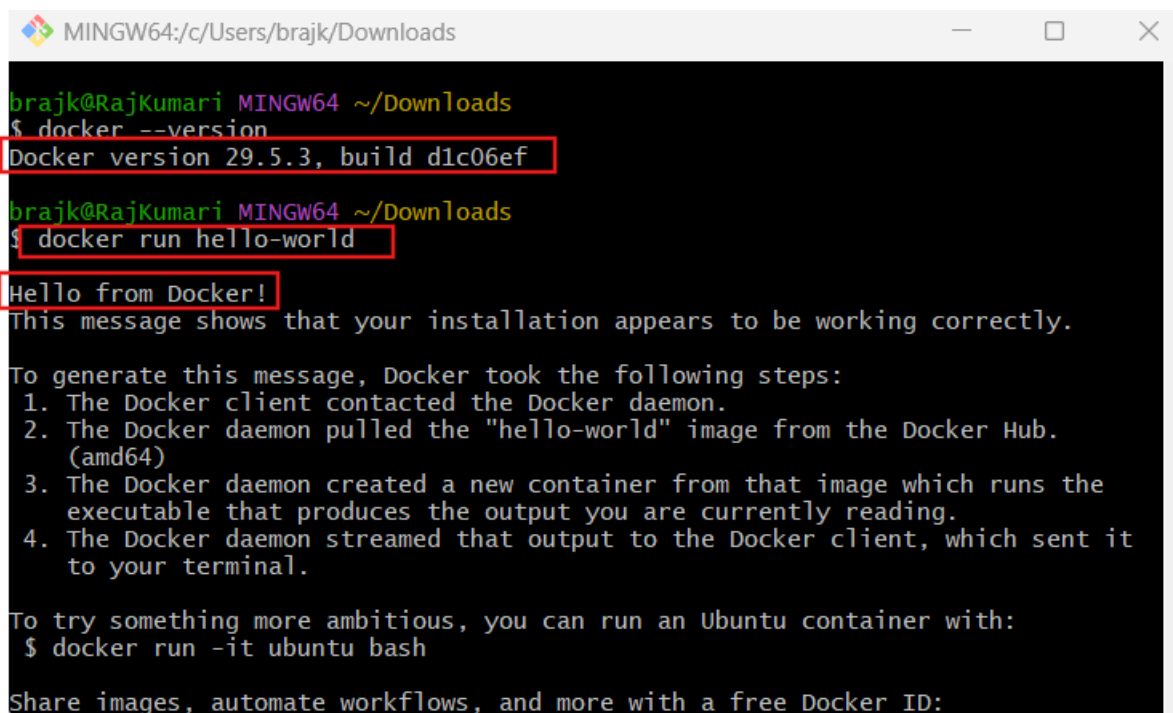
Also test:

`docker run hello-world`

You should get:

Hello from Docker!

If this works, Docker is ready.

A screenshot of a Windows terminal window titled 'MINGW64:/c/Users/brajK/Downloads'. The terminal shows the following commands and output:

```
brajk@RajKumari MINGW64 ~/Downloads
$ docker --version
Docker version 29.5.3, build d1c06ef

brajk@RajKumari MINGW64 ~/Downloads
$ docker run hello-world
Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
```

Step 4 — Install kubectl

kubectl is the command-line tool used to communicate with Kubernetes.

Download it from the official Kubernetes documentation:

[Install kubectl on Windows](#)

An easy method is using **winget**.

Open **PowerShell as Administrator**:

```
winget install -e --id Kubernetes.kubectl
```

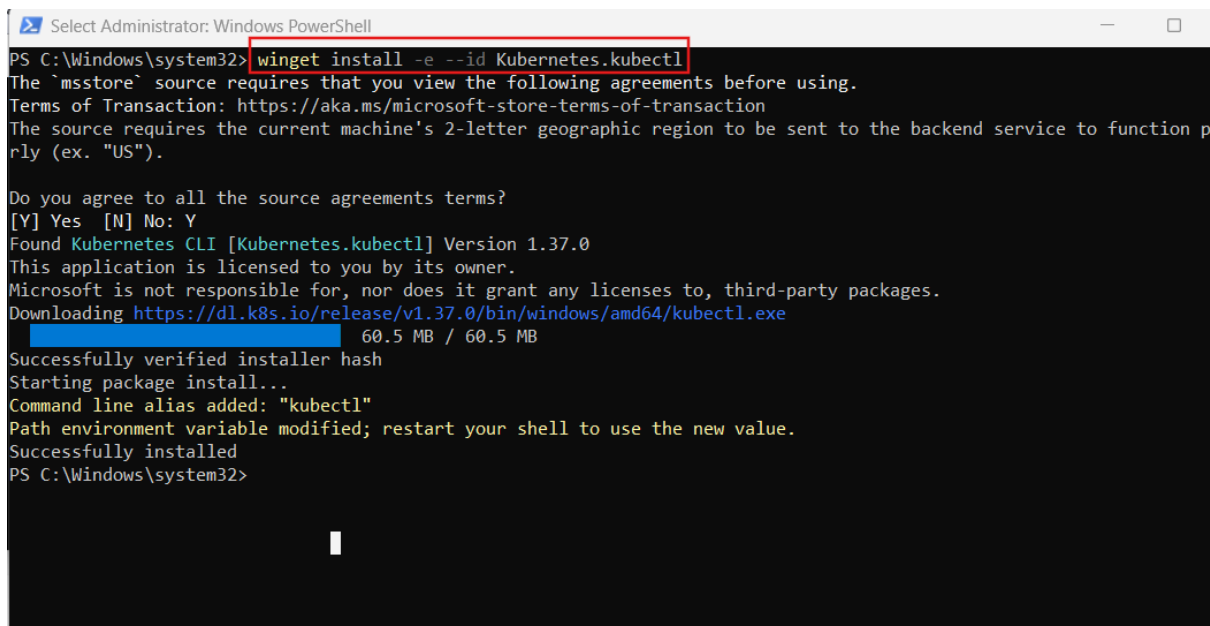
After installation, close and reopen your terminal.

Check:

kubectl version --client

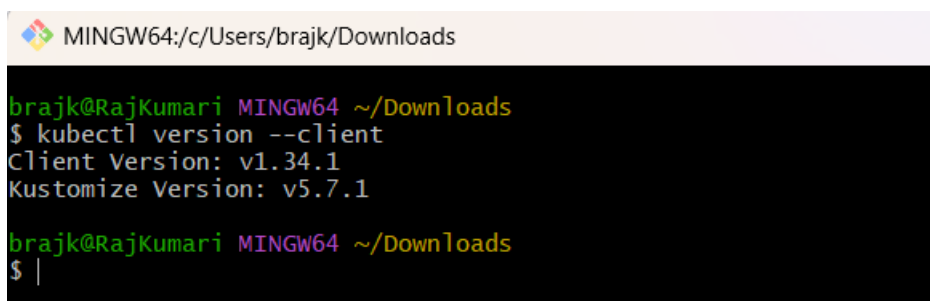
You should see something like:

Client Version: v1.xx.x



```
Select Administrator: Windows PowerShell
PS C:\Windows\system32> winget install -e --id Kubernetes.kubectl
The `msstore` source requires that you view the following agreements before using.
Terms of Transaction: https://aka.ms/microsoft-store-terms-of-transaction
The source requires the current machine's 2-letter geographic region to be sent to the backend service to function properly (ex. "US").

Do you agree to all the source agreements terms?
[Y] Yes [N] No: Y
Found Kubernetes CLI [Kubernetes.kubectl] Version 1.37.0
This application is licensed to you by its owner.
Microsoft is not responsible for, nor does it grant any licenses to, third-party packages.
Downloading https://dl.k8s.io/release/v1.37.0/bin/windows/amd64/kubectl.exe
60.5 MB / 60.5 MB
Successfully verified installer hash
Starting package install...
Command line alias added: "kubectl"
Path environment variable modified; restart your shell to use the new value.
Successfully installed
PS C:\Windows\system32>
```



```
MINGW64:/c/Users/brajK/Downloads
brajk@RajKumari MINGW64 ~/Downloads
$ kubectl version --client
Client Version: v1.34.1
Kustomize Version: v5.7.1
brajk@RajKumari MINGW64 ~/Downloads
$ |
```

Step 5 — Install Minikube

Download Minikube from the official Kubernetes/Minikube documentation:

[Minikube official documentation](#)

Using winget is convenient.

Open **PowerShell as Administrator**:

winget install Kubernetes.minikube

After installation, close and reopen Git Bash.

Check:

minikube version

You should get something similar to:

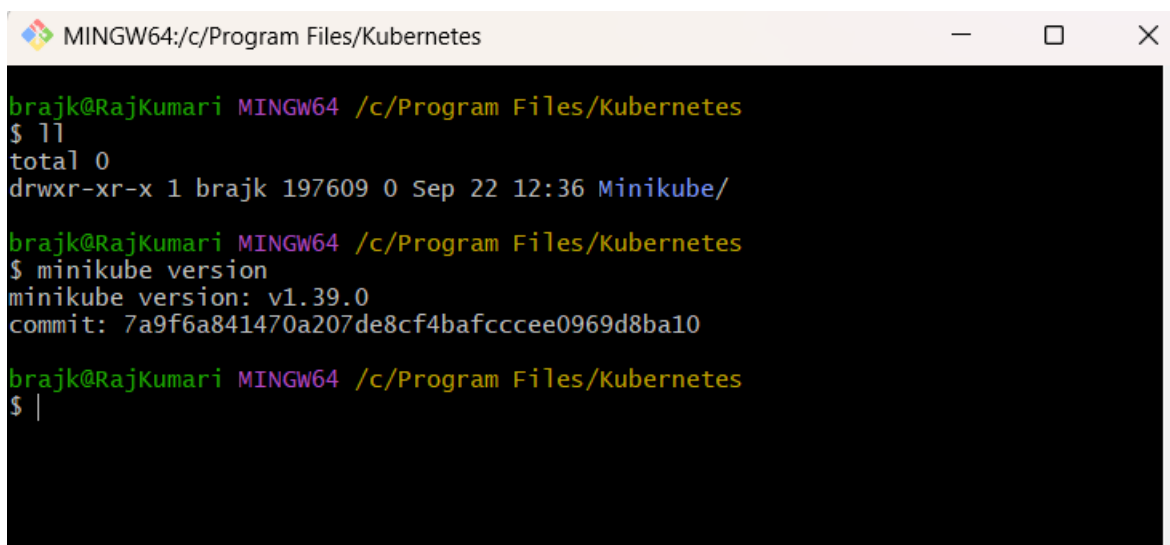
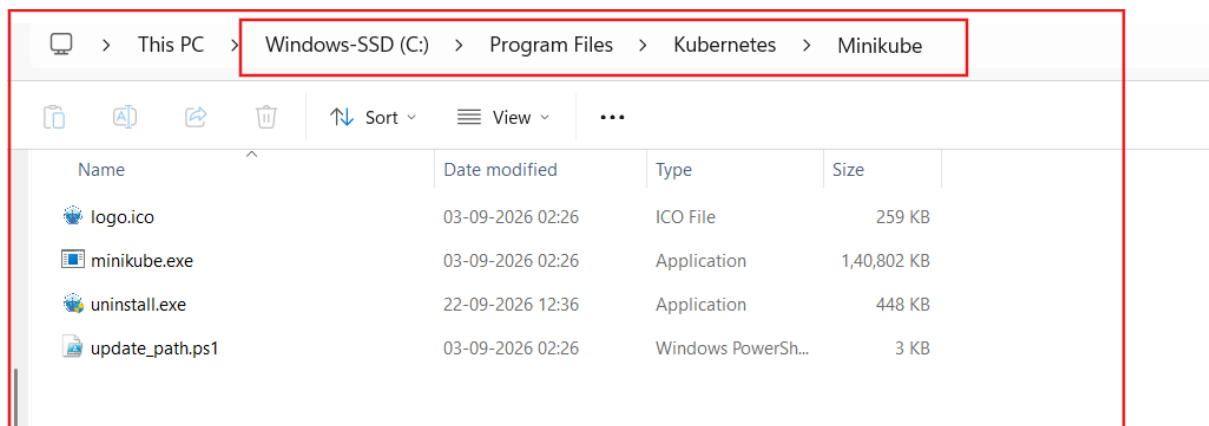
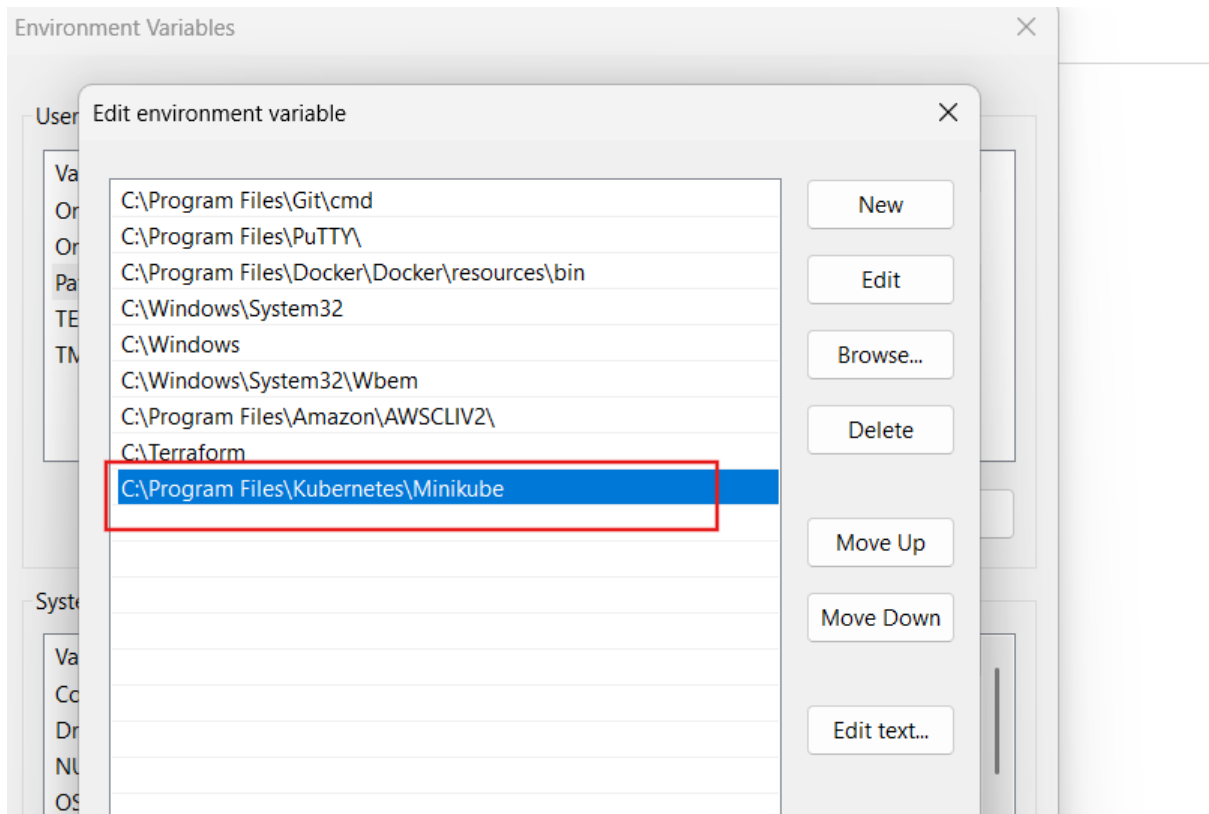
minikube version: v1.xx.x

```
Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Windows\system32> winget install Kubernetes.minikube
Found Kubernetes - Minikube - A Local Kubernetes Development Environment [Kubernetes.minikube] Version 1.39.0
This application is licensed to you by its owner.
Microsoft is not responsible for, nor does it grant any licenses to, third-party packages.
Downloading https://github.com/kubernetes/minikube/releases/download/v1.39.0/minikube-installer.exe
51.7 MB / 51.7 MB
Successfully verified installer hash
Starting package install...
Successfully installed
PS C:\Windows\system32>
```

```
Select Administrator: Windows PowerShell
PS C:\Windows\system32> winget list --id Kubernetes.minikube
Name Id Version Source
-----
Kubernetes - Minikube - A Local Kubernetes Development Environment Kubernetes.minikube 1.39.0 winget
PS C:\Windows\system32> winget show --id Kubernetes.minikube
Found Kubernetes - Minikube - A Local Kubernetes Development Environment [Kubernetes.minikube]
Version: 1.39.0
Publisher: Kubernetes
Publisher Url: https://github.com/kubernetes/minikube
Publisher Support Url: https://github.com/kubernetes/minikube/issues
Author: Kubernetes
Description: minikube implements a local Kubernetes cluster on macOS, Linux, and Windows. minikube's primary goals are to be the best tool for local Kubernetes application development and to support all Kubernetes features that fit.
Homepage: https://minikube.sigs.k8s.io/docs
License: Apache-2.0
License Url: https://github.com/kubernetes/minikube/blob/HEAD/LICENSE
Copyright: © 2022 The Kubernetes Authors -- All Rights Reserved
Copyright Url: https://github.com/kubernetes/minikube/blob/master/LICENSE
Release Notes:
👉 Please fill out our fast 5-question survey so that we can learn how & why you use minikube, and what improvements we should make. Thank you! 🙏
Release Notes
Version 1.39.0 - 2026-09-01
Highlights
- Add support for Kubernetes v1.36 and v1.37 (default: v1.37.0)
- Default container runtime is now containerd
- Add Traefik Helm-based ingress addon
- VirtualBox driver now supports Apple Silicon (VirtualBox 7.1+)
Features
```

Add Minikube to Windows PATH



Step 6 — Verify Docker Driver

Minikube will use Docker to create your Kubernetes cluster.

Run:

```
minikube config set driver docker
```

You should see:

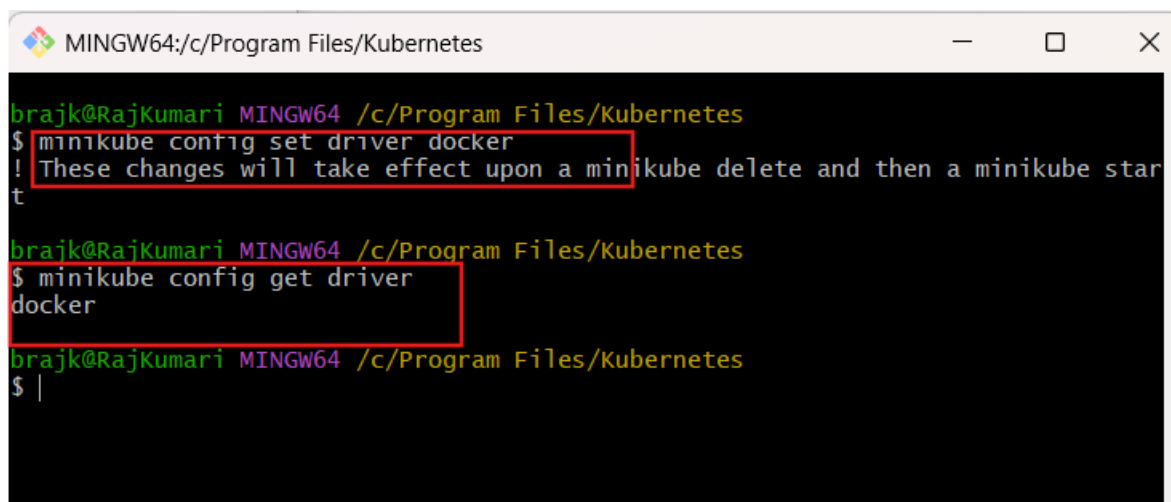
These changes will take effect immediately.

Verify:

```
minikube config get driver
```

Expected:

docker



```
MINGW64:/c/Program Files/Kubernetes
brajk@RajKumari MINGW64 /c/Program Files/Kubernetes
$ minikube config set driver docker
! These changes will take effect upon a minikube delete and then a minikube start
t
brajk@RajKumari MINGW64 /c/Program Files/Kubernetes
$ minikube config get driver
docker
brajk@RajKumari MINGW64 /c/Program Files/Kubernetes
$ |
```

Step 7 — Start Minikube

Now create your Kubernetes cluster:

```
minikube start --driver=docker
```

Minikube will:

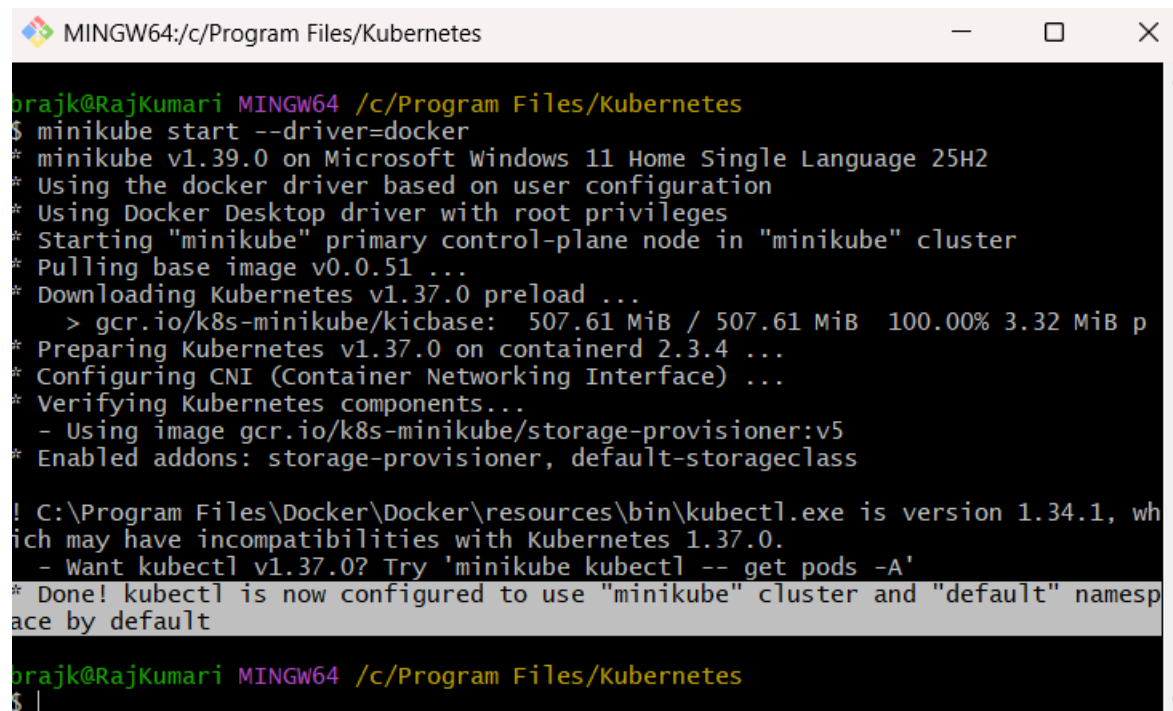
1. Download the required Kubernetes image
2. Create the Minikube container
3. Configure Kubernetes
4. Configure kubectl
5. Start the Kubernetes control plane

You should eventually see something similar to:

Done! kubectl is now configured to use "minikube" cluster

and "default" namespace by default

Your local Kubernetes cluster is now running.

A screenshot of a Windows command prompt window titled "MINGW64:/c/Program Files/Kubernetes". The window shows the execution of the 'minikube start --driver=docker' command. The output includes details about the minikube version (v1.39.0), the driver used (docker), and the process of starting the "minikube" cluster. It shows the download of the base image (v0.0.51) and the Kubernetes v1.37.0 preload. A warning message indicates that the installed kubectl version (1.34.1) is incompatible with the cluster's Kubernetes version (1.37.0) and suggests running 'minikube kubectl -- get pods -A' to verify. The final output states that kubectl is now configured to use the "minikube" cluster and the "default" namespace by default.

```
prajk@RajKumari MINGW64 /c/Program Files/Kubernetes
$ minikube start --driver=docker
* minikube v1.39.0 on Microsoft Windows 11 Home Single Language 25H2
* Using the docker driver based on user configuration
* Using Docker Desktop driver with root privileges
* Starting "minikube" primary control-plane node in "minikube" cluster
* Pulling base image v0.0.51 ...
* Downloading Kubernetes v1.37.0 preload ...
  > gcr.io/k8s-minikube/kicbase: 507.61 MiB / 507.61 MiB 100.00% 3.32 MiB p
* Preparing Kubernetes v1.37.0 on containerd 2.3.4 ...
* Configuring CNI (Container Networking Interface) ...
* Verifying Kubernetes components...
  - Using image gcr.io/k8s-minikube/storage-provisioner:v5
* Enabled addons: storage-provisioner, default-storageclass

! C:\Program Files\Docker\Docker\resources\bin\kubectl.exe is version 1.34.1, wh
ich may have incompatibilities with Kubernetes 1.37.0.
  - Want kubectl v1.37.0? Try 'minikube kubectl -- get pods -A'
* Done! kubectl is now configured to use "minikube" cluster and "default" namesp
ace by default

prajk@RajKumari MINGW64 /c/Program Files/Kubernetes
$
```

Step 8 — Check Minikube Status

Run:

```
minikube status
```

Expected output:

```
minikube
```

```
type: Control Plane
```

```
host: Running
```

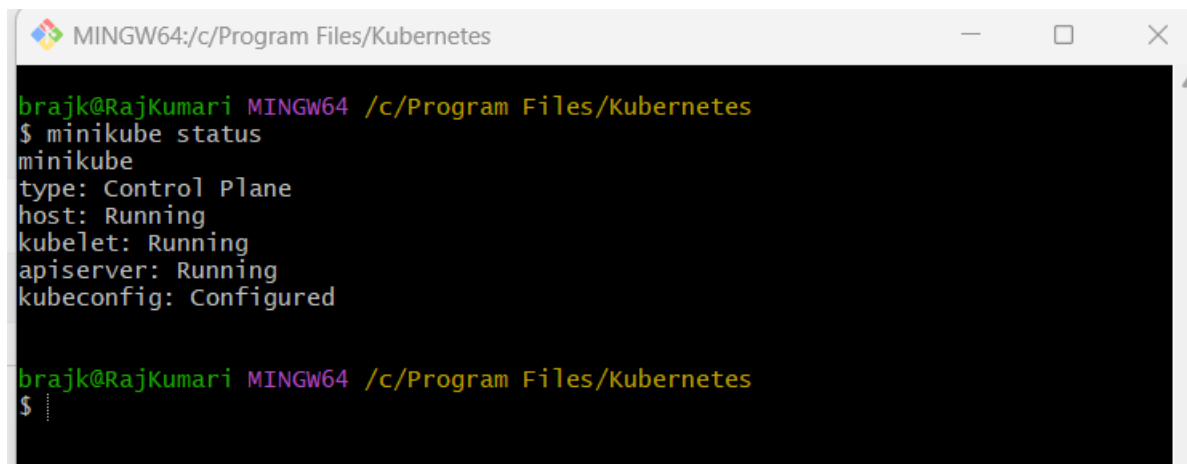
```
kubelet: Running
```

```
apiserver: Running
```

```
kubeconfig: Configured
```

All important components should show:

```
Running
```

A terminal window titled 'MINGW64:/c/Program Files/Kubernetes' showing the output of the 'minikube status' command. The output indicates that the minikube host, kubelet, apiserver, and kubeconfig are all in a ready or configured state.

```
brajk@RajKumari MINGW64 /c/Program Files/Kubernetes
$ minikube status
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured

brajk@RajKumari MINGW64 /c/Program Files/Kubernetes
$
```

Step 9 — Check Kubernetes Nodes

Run:

```
kubectl get nodes
```

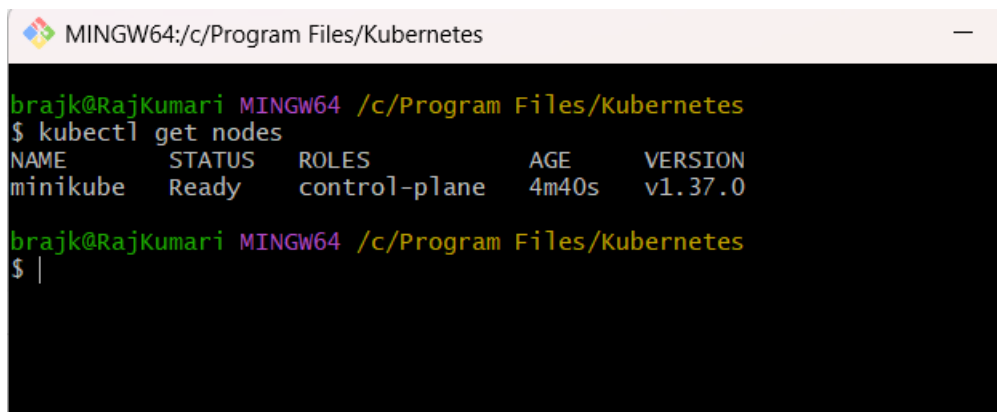
Expected:

NAME	STATUS	ROLES	AGE	VERSION
minikube	Ready	control-plane	...	v1.xx.x

The important part is:

STATUS

Ready

A terminal window titled 'MINGW64:/c/Program Files/Kubernetes' showing the output of the 'kubectl get nodes' command. The output shows a single node named 'minikube' with a status of 'Ready', role of 'control-plane', age of '4m40s', and version of 'v1.37.0'.

```
brajk@RajKumari MINGW64 /c/Program Files/Kubernetes
$ kubectl get nodes
NAME        STATUS    ROLES    AGE   VERSION
minikube    Ready    control-plane  4m40s  v1.37.0

brajk@RajKumari MINGW64 /c/Program Files/Kubernetes
$ |
```

Step 10 — Check Kubernetes Pods

Run:

```
kubectl get pods -A
```

You should see Kubernetes system pods such as:

NAMESPACE NAME

kube-system coredns-...

kube-system etcd-minikube

kube-system kube-apiserver-minikube

kube-system kube-controller-manager-minikube

kube-system kube-proxy-...

kube-system kube-scheduler-minikube

They should eventually show:

Running

```
MINGW64/c/Program Files/Kubernetes
brajk@RajKumari MINGW64 /c/Program Files/Kubernetes
$ kubectl get pods -A
NAMESPACE NAME READY STATUS RESTARTS AG
E
kube-system coredns-559f6c778d-tbdrv 1/1 Running 0 5m
29s
kube-system etcd-minikube 1/1 Running 0 5m
37s
kube-system kindnet-k6268 1/1 Running 0 5m
29s
kube-system kube-apiserver-minikube 1/1 Running 0 5m
37s
kube-system kube-controller-manager-minikube 1/1 Running 0 5m
37s
kube-system kube-proxy-kwt42 1/1 Running 0 5m
29s
kube-system kube-scheduler-minikube 1/1 Running 0 5m
37s
kube-system storage-provisioner 1/1 Running 0 5m
34s
brajk@RajKumari MINGW64 /c/Program Files/Kubernetes
$ |
```

Step 11 — Check Cluster Information

Run:

kubectl cluster-info

You should see something similar to:

Kubernetes control plane is running at https://...

CoreDNS is running at https://...

```
MINGW64:/c/Program Files/Kubernetes
brajk@RajKumari MINGW64 /c/Program Files/Kubernetes
$ kubectl cluster-info
Kubernetes control plane is running at https://127.0.0.1:61635
CoreDNS is running at https://127.0.0.1:61635/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
brajk@RajKumari MINGW64 /c/Program Files/Kubernetes
$
```

Step 12 — Check Minikube Docker Container

Because we're using Docker as the driver, run:

```
docker ps
```

You should see a container related to Minikube.

For example:

CONTAINER ID	IMAGE	NAMES
xxxxxxx	gcr.io/k8s-minikube/minikube	minikube

This demonstrates an important concept:

Docker

|

+--- Minikube container

|

+--- Kubernetes

|

+--- API Server

+--- Scheduler

+--- Controller Manager

+--- etcd

```
brajk@RajKumari MINGW64 /c/Program Files/Kubernetes
$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CR
127.0.0.1:61632->5000/tcp, 127.0.0.1:61635->8443/tcp
brajk@RajKumari MINGW64 /c/Program Files/Kubernetes
$ docker ps
CONTAINER ID   IMAGE                                NAMES                    COMMAND                  CREATED      STATUS      PORTS
127.0.0.1:61632->5000/tcp, 127.0.0.1:61633->2376/tcp, 127.0.0.1:61632->5000/tcp, 127.0.0.1:61635->8443/tcp
```

Step 13 — Create Your First Kubernetes Deployment

Now let's test whether Kubernetes is actually working.

Run:

```
kubectl create deployment nginx --image=nginx
```

Expected:

```
deployment.apps/nginx created
```

Check the deployment:

```
kubectl get deployments
```

Expected:

```
NAME    READY    UP-TO-DATE    AVAILABLE
```

```
nginx   1/1      1             1
```

```
brajk@RajKumari MINGW64 /c/Program Files/Kubernetes
$ kubectl create deployment nginx --image=nginx
deployment.apps/nginx created

brajk@RajKumari MINGW64 /c/Program Files/Kubernetes
$ kubectl get deployments
NAME    READY    UP-TO-DATE    AVAILABLE    AGE
nginx   0/1      1             0            10s

brajk@RajKumari MINGW64 /c/Program Files/Kubernetes
$ |
```

Step 14 — Check the Pod

Run:

kubectl get pods

Expected:

NAME	READY	STATUS	RESTARTS	AGE
------	-------	--------	----------	-----

nginx-xxxxxxxxxx-xxxxx	1/1	Running	0	...
------------------------	-----	---------	---	-----

The important thing is:

STATUS

Running

```
MINGW64:/c/Program Files/Kubernetes  
brajk@RajKumari MINGW64 /c/Program Files/Kubernetes  
$ kubectl create deployment nginx --image=nginx  
deployment.apps/nginx created  
  
brajk@RajKumari MINGW64 /c/Program Files/Kubernetes  
$ kubectl get deployments  
NAME      READY   UP-TO-DATE   AVAILABLE   AGE  
nginx     0/1     1            0           10s  
  
brajk@RajKumari MINGW64 /c/Program Files/Kubernetes  
$ kubectl get pods  
NAME                  READY   STATUS    RESTARTS   AGE  
nginx-69b9cdbbdd-xxqpk 1/1     Running   0           79s  
  
brajk@RajKumari MINGW64 /c/Program Files/Kubernetes  
$ |
```

Step 15 — Expose Nginx

Create a Kubernetes Service:

kubectl expose deployment nginx --type=NodePort --port=80

Expected:

service/nginx exposed

Check:

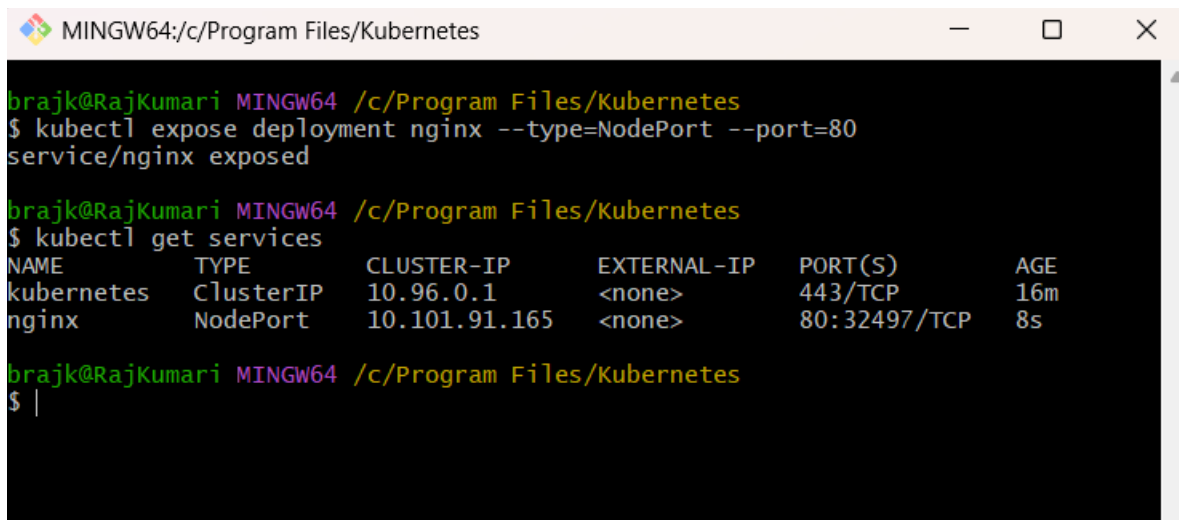
kubectl get services

You should see:

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
------	------	------------	-------------	---------

kubernetes ClusterIP ...

nginx NodePort ... <none> 80:xxxxx/TCP



```
MINGW64:/c/Program Files/Kubernetes
brajk@RajKumari MINGW64 /c/Program Files/Kubernetes
$ kubectl expose deployment nginx --type=NodePort --port=80
service/nginx exposed

brajk@RajKumari MINGW64 /c/Program Files/Kubernetes
$ kubectl get services
NAME         TYPE        CLUSTER-IP    EXTERNAL-IP  PORT(S)          AGE
kubernetes   ClusterIP   10.96.0.1     <none>       443/TCP          16m
nginx        NodePort    10.101.91.165 <none>       80:32497/TCP     8s

brajk@RajKumari MINGW64 /c/Program Files/Kubernetes
$ |
```

Step 16 — Open Nginx

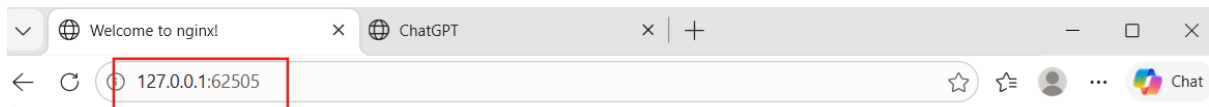
The easiest way with Minikube is:

minikube service nginx

Minikube will open the Nginx page in your browser.

You should see:

Welcome to nginx!

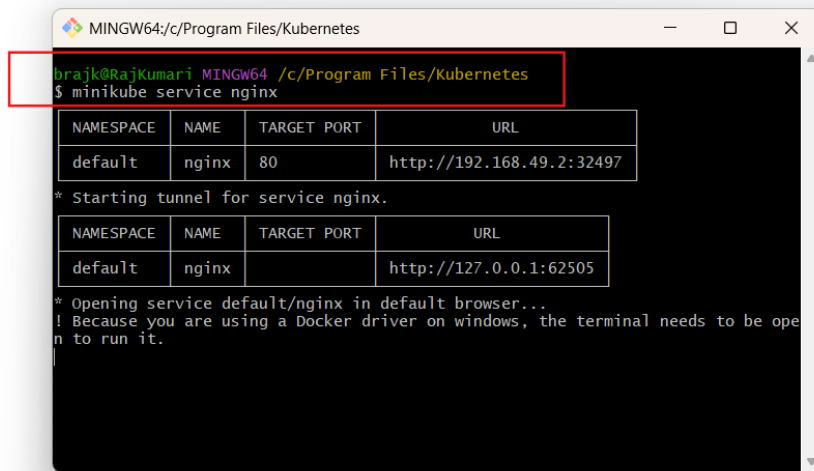


Welcome to nginx!

If you see this page, nginx is successfully installed and working. Further configuration is required for the web server, reverse proxy, API gateway, load balancer, content cache, or other features.

For online documentation and support please refer to nginx.org.
To engage with the community please visit community.nginx.org.
For enterprise grade support, professional services, additional security features and capabilities please refer to f5.com/nginx.

Thank you for using nginx.



Step 17 — Check Minikube Dashboard

Minikube includes a Kubernetes Dashboard.

Run:

minikube dashboard

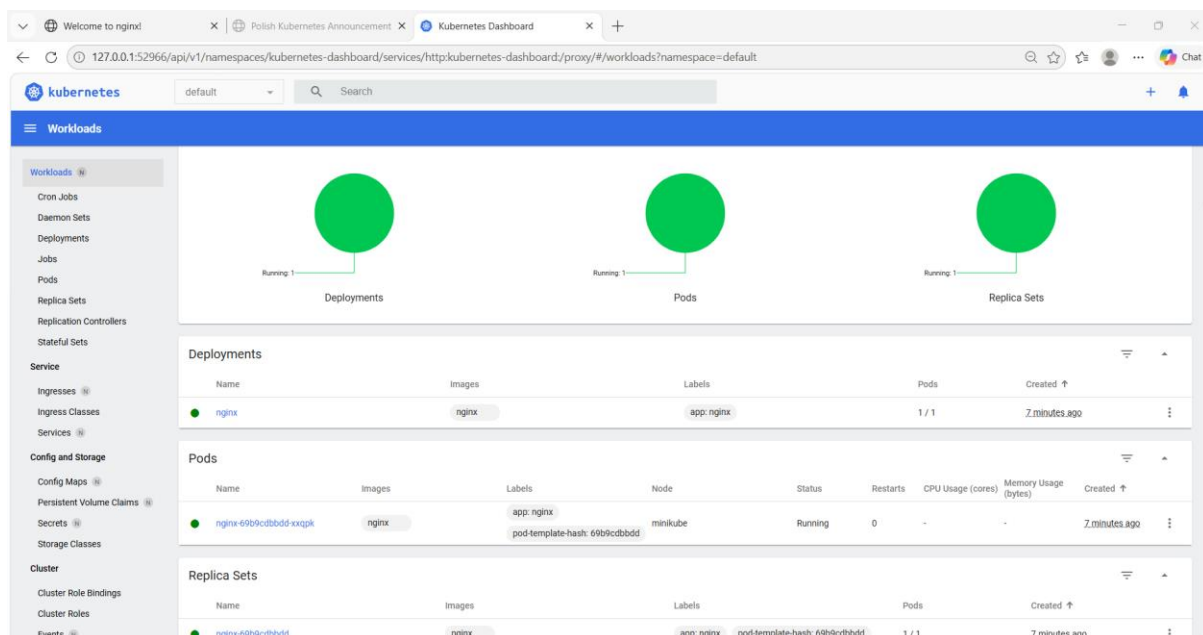
```

brajk@RajKumari MINGW64 /c/Program Files/Kubernetes
$ minikube dashboard
* Enabling dashboard ...
  - Using image docker.io/kubernetesui/dashboard:v2.7.0
  - Using image docker.io/kubernetesui/metrics-scraper:v1.0.8
* Some dashboard features require the metrics-server addon. To enable all features please run:

    minikube addons enable metrics-server

* Verifying dashboard health ...
* Launching proxy ...
* Verifying proxy health ...
* Opening http://127.0.0.1:52966/api/v1/namespaces/kubernetes-dashboard/services/http:kubernetes-dashboard:/proxy/ in your default browser...

```



Step 18 – Useful Minikube Commands

Start Minikube

minikube start

Stop Minikube

minikube stop

Start again

minikube start

Check status

minikube status

See Minikube version

`minikube version`

Delete cluster

`minikube delete`

⚠ `minikube delete` removes your Minikube cluster.

SSH into Minikube

`minikube ssh`

View Kubernetes nodes

`kubectl get nodes`

View all pods

`kubectl get pods -A`

View services

`kubectl get svc`

View deployments

`kubectl get deployments`

Step 19 — Verify Everything

At the end, run these commands one by one:

`docker version`

`kubectl version --client`

`minikube version`

`minikube status`

`kubectl get nodes`

`kubectl get pods -A`

`kubectl cluster-info`

You want to reach this state:

Docker → Working

kubectl → Installed

Minikube → Running

Kubernetes → Running

Node → Ready

Pods → Running

□ Complete Command Cheat Sheet

Once Docker Desktop is installed, the core setup can be done with:

`docker version`

`kubectl version --client`

`minikube version`

`minikube config set driver docker`

`minikube start --driver=docker`

`minikube status`

`kubectl get nodes`

`kubectl get pods -A`

`kubectl cluster-info`

Then test an application:

`kubectl create deployment nginx --image=nginx`

`kubectl get deployments`

`kubectl get pods`

`kubectl expose deployment nginx --type=NodePort --port=80`

`kubectl get svc`

`minikube service nginx`

2. Setup k8s master and two worker nodes on ubuntu.

Kubernetes Cluster Setup: 1 Master + 2 Worker Nodes on Ubuntu

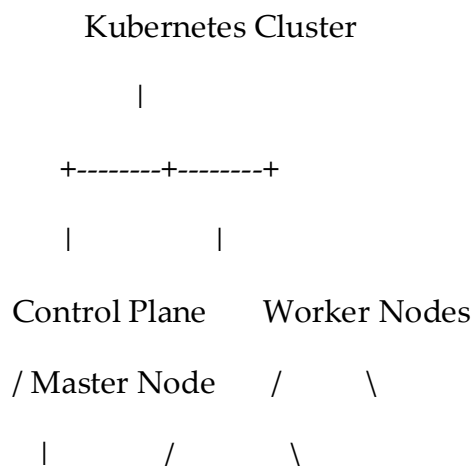
Below **setup** for creating a Kubernetes cluster with:

- **1 Master/Control Plane**
- **2 Worker Nodes**
- **Ubuntu 22.04/24.04**
- **containerd** as the container runtime
- **kubeadm** for cluster creation
- **kubectl** for cluster administration
- **Calico** as the CNI/network plugin

1. Objective

The objective of this exercise is to create a **multi-node Kubernetes cluster** on Ubuntu.

At the end, you should have:



kube-apiserver	Worker-1	Worker-2
kube-scheduler		
controller-manager	Pods	Pods
etcd		

We will configure:

Node	Hostname	Example Private IP	Role
Master	k8s-master	10.0.1.10	Control Plane
Worker 1	k8s-worker1	10.0.1.11	Worker
Worker 2	k8s-worker2	10.0.1.12	Worker

Important: Replace these example IP addresses with your actual private IP addresses.

2. Prerequisites

Before starting, make sure you have:

AWS requirements

- AWS account
- AWS region
- VPC
- Internet Gateway
- Public subnet
- Security Group
- 3 Ubuntu EC2 instances
- Key pair

Recommended EC2 configuration

Node	Instance	OS	Purpose
Master	t3.medium	Ubuntu 24.04 LTS	Control Plane

Node	Instance	OS	Purpose
------	----------	----	---------

Worker 1	t3.medium	Ubuntu 24.04 LTS	Worker
----------	-----------	------------------	--------

Worker 2	t3.medium	Ubuntu 24.04 LTS	Worker
----------	-----------	------------------	--------

For a classroom/lab environment, smaller instances may work, but t3.medium gives you more room for Kubernetes components.

3. Create 3 EC2 Instances

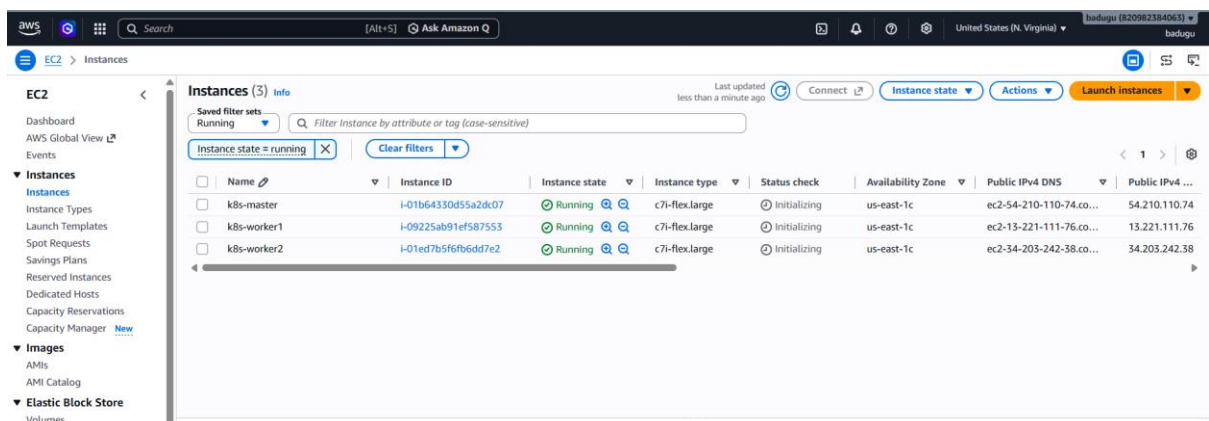
Create three Ubuntu EC2 instances.

Use meaningful names:

k8s-master

k8s-worker1

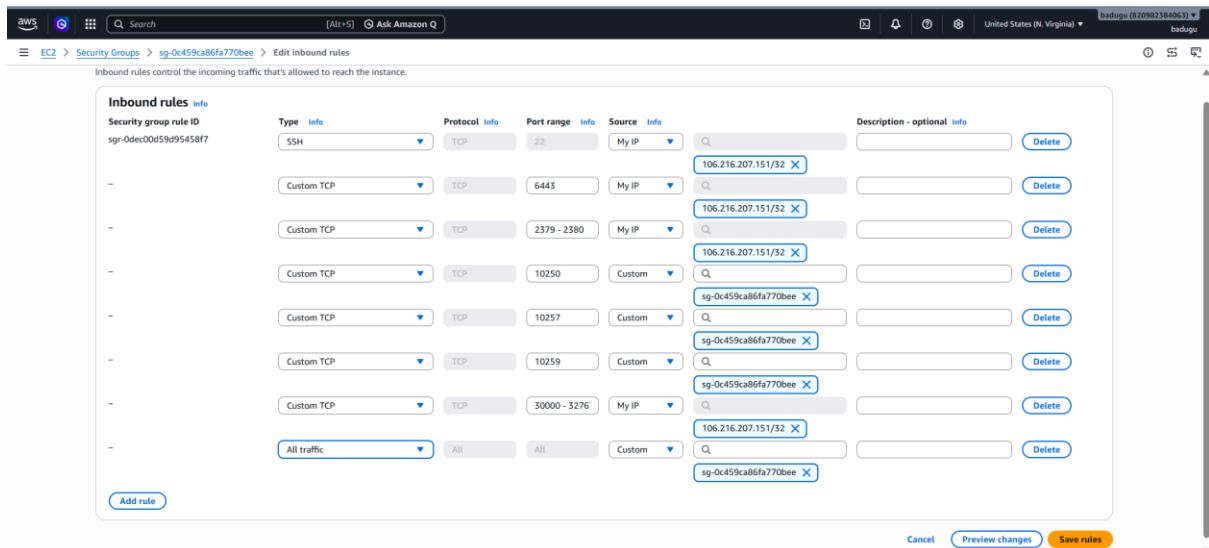
k8s-worker2



4. Configure AWS Security Group

Create a security group such as:

k8s-cluster-sg



5. Connect to the Master Node

From Windows Git Bash:

```
ssh -i ~/Downloads/docker.pem ubuntu@<MASTER_PUBLIC_IP>
```

Example:

```
ssh -i ~/Downloads/docker.pem ubuntu@54.x.x.x
```

Verify:

Hostname

```
ubuntu@ip-172-31-17-127: ~
ubuntu@ip-172-31-17-127:~$ hostname
ip-172-31-17-127
ubuntu@ip-172-31-17-127:~$ sudo apt update
sudo apt upgrade -y
Hit:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute
```

```

ubuntu@ip-172-31-17-127: ~
brajk@Rajkumari MINGW64 ~/Downloads
$ ssh -i docker.pem ubuntu@54.210.110.74
The authenticity of host '54.210.110.74 (54.210.110.74)' can't be established.
ED25519 key fingerprint is SHA256:eyONkygxEBCE43rLCvZNSivpQiwPZkorr3Rc/f7AGAs.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '54.210.110.74' (ED25519) to the list of known hosts.
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-1006-aws x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Tue Sep 22 08:32:18 UTC 2026

System load:  0.08           Temperature:   -273.1 C
Usage of /:   30.5% of 6.61GB Processes:      153
Memory usage: 6%            Users logged in: 0
Swap usage:   0%            IPv4 address for enp39s0: 172.31.17.127

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.
ubuntu@ip-172-31-17-127:~$

```

6. Update Ubuntu

Run this on **all three nodes**.

```
sudo apt update
```

```
sudo apt upgrade -y
```

You can verify the Ubuntu version:

```
cat /etc/os-release
```

```
ubuntu@ip-172-31-17-127: ~  
No containers need to be restarted.  
  
User sessions running outdated binaries:  
ubuntu @ session #1: apt[1568], bash[1337], sshd-session[1217,1334],  
sudo[1565,1567]  
ubuntu @ user manager: (sd-pam)[1224]  
  
No VM guests are running outdated hypervisor (qemu) binaries on this host.  
ubuntu@ip-172-31-17-127:~$ cat /etc/os-release  
PRETTY_NAME="Ubuntu 26.04.1 LTS"  
NAME="Ubuntu"  
VERSION_ID="26.04"  
VERSION="26.04.1 LTS (Resolute Raccoon)"  
VERSION_CODENAME=resolute  
ID=ubuntu  
ID_LIKE=debian  
HOME_URL="https://www.ubuntu.com/"  
SUPPORT_URL="https://help.ubuntu.com/"  
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"  
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"  
UBUNTU_CODENAME=resolute  
LOGO=ubuntu-logo  
ubuntu@ip-172-31-17-127:~$
```

7. Set Hostnames

On Master

`sudo hostnamectl set-hostname k8s-master`

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ |
```

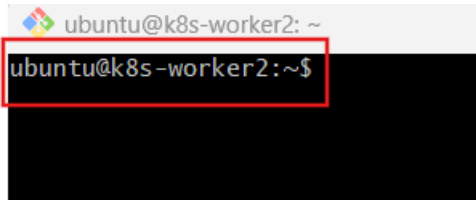
On Worker 1

`sudo hostnamectl set-hostname k8s-worker1`

```
ubuntu@k8s-worker1: ~  
ubuntu@k8s-worker1:~$ |
```

On Worker 2

`sudo hostnamectl set-hostname k8s-worker2`



8. Configure /etc/hosts

We want all nodes to be able to resolve each other.

First get private IPs:

hostname -I

Example:

172.31.20.10 k8s-master

172.31.20.11 k8s-worker1

172.31.20.12 k8s-worker2

On **all three nodes**, edit:

sudo nano /etc/hosts

Add:

172.31.20.10 k8s-master

172.31.20.11 k8s-worker1

172.31.20.12 k8s-worker2

Replace the IP addresses with your actual private IPs.

Test:

ping -c 3 k8s-master

ping -c 3 k8s-worker1

ping -c 3 k8s-worker2

Easier method — add entries without Nano

You can also use:

sudo tee -a /etc/hosts <<EOF

172.31.20.10 k8s-master

172.31.20.11 k8s-worker1

172.31.20.12 k8s-worker2

EOF

```
ubuntu@k8s-master: ~  
127.0.0.1 localhost  
172.31.17.127 k8s-master  
172.31.16.155 k8s-worker1  
172.31.26.51 k8s-worker2  
  
# The following lines are desirable for IPv6 capable hosts  
::1 ip6-localhost ip6-loopback  
fe00::0 ip6-localnet  
ff00::0 ip6-mcastprefix  
ff02::1 ip6-allnodes  
ff02::2 ip6-allrouters  
ff02::3 ip6-allhosts  
~  
~  
~  
~  
~
```

```

ubuntu@k8s-master: ~
ubuntu@k8s-master:~$ sudo tee -a /etc/hosts <<EOF
172.31.17.127 k8s-master
172.31.16.155 k8s-worker1
172.31.26.51 k8s-worker2
EOF
172.31.17.127 k8s-master
172.31.16.155 k8s-worker1
172.31.26.51 k8s-worker2
ubuntu@k8s-master:~$ ping -c 3 k8s-master
PING k8s-master (172.31.17.127) 56(84) bytes of data.
64 bytes from k8s-master (172.31.17.127): icmp_seq=1 ttl=64 time=0.022 ms
64 bytes from k8s-master (172.31.17.127): icmp_seq=2 ttl=64 time=0.036 ms
64 bytes from k8s-master (172.31.17.127): icmp_seq=3 ttl=64 time=0.044 ms

--- k8s-master ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2046ms
rtt min/avg/max/mdev = 0.022/0.034/0.044/0.009 ms
ubuntu@k8s-master:~$ ping -c 3 k8s-worker1
PING k8s-worker1 (172.31.16.155) 56(84) bytes of data.
64 bytes from k8s-worker1 (172.31.16.155): icmp_seq=1 ttl=64 time=1.98 ms
64 bytes from k8s-worker1 (172.31.16.155): icmp_seq=2 ttl=64 time=0.608 ms
64 bytes from k8s-worker1 (172.31.16.155): icmp_seq=3 ttl=64 time=0.419 ms

--- k8s-worker1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2052ms
rtt min/avg/max/mdev = 0.419/1.001/1.978/0.694 ms
ubuntu@k8s-master:~$ ping -c 3 k8s-worker2
PING k8s-worker2 (172.31.26.51) 56(84) bytes of data.
64 bytes from k8s-worker2 (172.31.26.51): icmp_seq=1 ttl=64 time=1.28 ms
64 bytes from k8s-worker2 (172.31.26.51): icmp_seq=2 ttl=64 time=0.309 ms
64 bytes from k8s-worker2 (172.31.26.51): icmp_seq=3 ttl=64 time=0.350 ms

--- k8s-worker2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2016ms
rtt min/avg/max/mdev = 0.309/0.647/1.282/0.449 ms
ubuntu@k8s-master:~$ |

```

9. Disable Swap

Kubernetes requires swap to be disabled for the standard kubeadm setup.

Run on **all nodes**:

```
sudo swapoff -a
```

Check:

```
free -h
```

Swap should show:

```
Swap: 0B 0B 0B
```

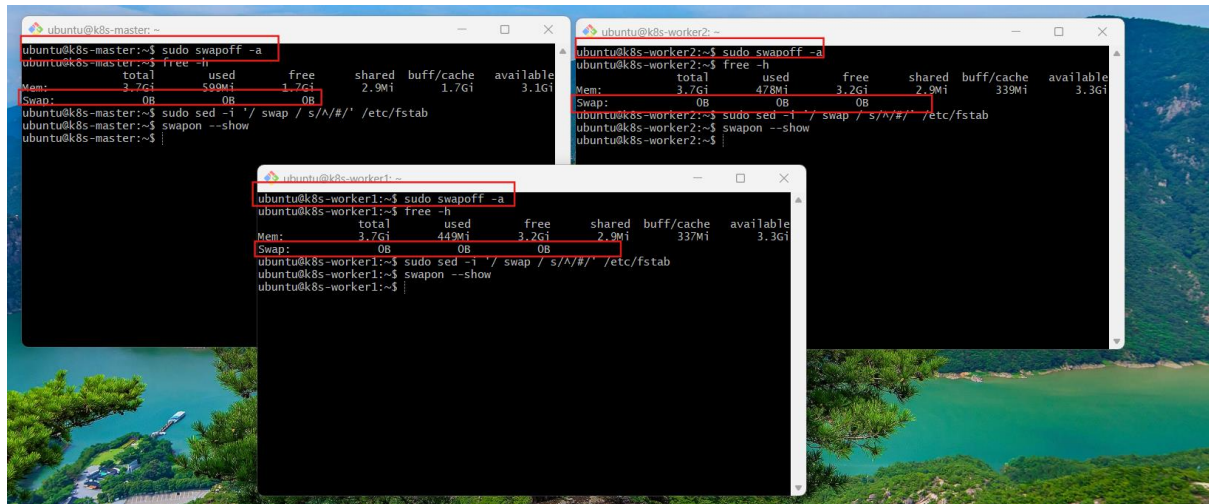
Now permanently disable swap.

```
sudo sed -i 's/^#/' /etc/fstab
```

Verify:

`swapon --show`

There should be no output.



10. Load Required Kernel Modules

Run on **all three nodes**:

`sudo modprobe overlay`

`sudo modprobe br_netfilter`

Make them persistent:

`cat <<EOF | sudo tee /etc/modules-load.d/k8s.conf`

`overlay`

`br_netfilter`

`EOF`

Verify:

`lsmod | grep br_netfilter`

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ sudo modprobe overlay  
sudo modprobe br_netfilter  
ubuntu@k8s-master:~$ cat <<EOF | sudo tee /etc/modules-load.d/k8s.conf  
overlay  
br_netfilter  
EOF  
ubuntu@k8s-master:~$ lsmod | grep br_netfilter  
br_netfilter 32768 0  
bridge 430080 1 br_netfilter  
ubuntu@k8s-master:~$ |
```

```
ubuntu@k8s-worker1: ~  
ubuntu@k8s-worker1:~$ sudo modprobe overlay  
sudo modprobe br_netfilter  
ubuntu@k8s-worker1:~$ cat <<EOF | sudo tee /etc/modules-load.d/k8s.conf  
overlay  
br_netfilter  
EOF  
ubuntu@k8s-worker1:~$ lsmod | grep br_netfilter  
br_netfilter 32768 0  
bridge 430080 1 br_netfilter  
ubuntu@k8s-worker1:~$ |
```

```
ubuntu@k8s-worker2: ~  
ubuntu@k8s-worker2:~$ sudo modprobe overlay  
sudo modprobe br_netfilter  
ubuntu@k8s-worker2:~$ cat <<EOF | sudo tee /etc/modules-load.d/k8s.conf  
overlay  
br_netfilter  
EOF  
overlay  
br_netfilter  
ubuntu@k8s-worker2:~$ lsmod | grep br_netfilter  
br_netfilter      32768  0  
bridge           430080  1 br_netfilter  
ubuntu@k8s-worker2:~$
```

11. Configure Kubernetes Network Parameters

Run on **all nodes**:

```
cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
```

```
net.bridge.bridge-nf-call-iptables = 1
```

```
net.bridge.bridge-nf-call-ip6tables = 1
```

```
net.ipv4.ip_forward = 1
```

```
EOF
```

Apply:

```
sudo sysctl --system
```

Verify:

```
sysctl net.ipv4.ip_forward
```

Expected:

```
net.ipv4.ip_forward = 1
```

```
ubuntu@k8s-master: ~  
kernel.apparmor_restrict_unprivileged_userns = 1  
kernel.apparmor_restrict_unprivileged_unconfined = 1  
kernel.core_pattern = core  
net.ipv4.neigh.default.gc_thresh2 = 15360  
net.ipv4.neigh.default.gc_thresh3 = 16384  
net.netfilter.nf_conntrack_max = 1048576  
kernel.pid_max = 4194304  
net.core.default_qdisc = fq_codel  
kernel.printk = 4 4 1 7  
net.ipv6.conf.all.use_tempaddr = 2  
net.ipv6.conf.default.use_tempaddr = 2  
kernel.kptr_restrict = 1  
kernel.sysrq = 176  
vm.max_map_count = 1048576  
net.ipv4.conf.default.rp_filter = 2  
net.ipv4.conf.all.rp_filter = 2  
kernel.yama.ptrace_scope = 1  
vm.mmap_min_addr = 65536  
net.ipv6.conf.all.use_tempaddr = 0  
net.ipv6.conf.default.use_tempaddr = 0  
net.bridge.bridge-nf-call-iptables = 1  
net.bridge.bridge-nf-call-ip6tables = 1  
net.ipv4.ip_forward = 1  
ubuntu@k8s-master:~$
```

```
ubuntu@k8s-worker1: ~  
kernel.apparmor_restrict_unprivileged_userns = 1  
kernel.apparmor_restrict_unprivileged_unconfined = 1  
kernel.core_pattern = core  
net.ipv4.neigh.default.gc_thresh2 = 15360  
net.ipv4.neigh.default.gc_thresh3 = 16384  
net.netfilter.nf_conntrack_max = 1048576  
kernel.pid_max = 4194304  
net.core.default_qdisc = fq_codel  
kernel.printk = 4 4 1 7  
net.ipv6.conf.all.use_tempaddr = 2  
net.ipv6.conf.default.use_tempaddr = 2  
kernel.kptr_restrict = 1  
kernel.sysrq = 176  
vm.max_map_count = 1048576  
net.ipv4.conf.default.rp_filter = 2  
net.ipv4.conf.all.rp_filter = 2  
kernel.yama.ptrace_scope = 1  
vm.mmap_min_addr = 65536  
net.ipv6.conf.all.use_tempaddr = 0  
net.ipv6.conf.default.use_tempaddr = 0  
net.bridge.bridge-nf-call-iptables = 1  
net.bridge.bridge-nf-call-ip6tables = 1  
net.ipv4.ip_forward = 1  
ubuntu@k8s-worker1:~$
```

```
ubuntu@k8s-worker2: ~  
kernel.apparmor_restrict_unprivileged_userns = 1  
kernel.apparmor_restrict_unprivileged_unconfined = 1  
kernel.core_pattern = core  
net.ipv4.neigh.default.gc_thresh2 = 15360  
net.ipv4.neigh.default.gc_thresh3 = 16384  
net.netfilter.nf_conntrack_max = 1048576  
kernel.pid_max = 4194304  
net.core.default_qdisc = fq_codel  
kernel.printk = 4 4 1 7  
net.ipv6.conf.all.use_tempaddr = 2  
net.ipv6.conf.default.use_tempaddr = 2  
kernel.kptr_restrict = 1  
kernel.sysrq = 176  
vm.max_map_count = 1048576  
net.ipv4.conf.default.rp_filter = 2  
net.ipv4.conf.all.rp_filter = 2  
kernel.yama.ptrace_scope = 1  
vm.mmap_min_addr = 65536  
net.ipv6.conf.all.use_tempaddr = 0  
net.ipv6.conf.default.use_tempaddr = 0  
net.bridge.bridge-nf-call-iptables = 1  
net.bridge.bridge-nf-call-ip6tables = 1  
net.ipv4.ip_forward = 1  
ubuntu@k8s-worker2:~$
```

12. Install containerd

Kubernetes needs a container runtime.

We will use:

containerd

Run on **all three nodes**:

sudo apt update

sudo apt install -y containerd

Check:

containerd --version

```
ubuntu@k8s-master: ~  
7.0.0-1012-aws.  
  
Restarting the system to load the new kernel will not be handled automatically,  
so you should consider rebooting.  
  
Restarting services...  
  
Service restarts being deferred:  
/etc/needrestart/restart.d/dbus.service  
systemctl restart getty@tty1.service  
systemctl restart networkd-dispatcher.service  
systemctl restart serial-getty@ttyS0.service  
systemctl restart systemd-logind.service  
systemctl restart unattended-upgrades.service  
  
No containers need to be restarted.  
  
User sessions running outdated binaries:  
ubuntu @ user manager: (sd-pam)[1224]  
  
No VM guests are running outdated hypervisor (qemu) binaries on this host.  
ubuntu@k8s-master:~$ containerd --version  
containerd github.com/containerd/containerd/v2 2.2.2  
ubuntu@k8s-master:~$
```

```
ubuntu@k8s-worker1: ~  
Unpacking runc (1.4.0-0ubuntu1) ...  
Selecting previously unselected package containerd.  
Preparing to unpack .../containerd_2.2.2-0ubuntu1.1_amd64v3.deb ...  
Unpacking containerd (2.2.2-0ubuntu1.1) ...  
Setting up runc (1.4.0-0ubuntu1) ...  
Setting up containerd (2.2.2-0ubuntu1.1) ...  
Created symlink '/etc/systemd/system/multi-user.target.wants/containerd.service'  
→ '/usr/lib/systemd/system/containerd.service'.  
Processing triggers for man-db (2.13.1-1build1) ...  
Scanning processes...  
Scanning linux images...  
  
Running kernel seems to be up-to-date.  
  
No services need to be restarted.  
  
No containers need to be restarted.  
  
No user sessions are running outdated binaries.  
  
No VM guests are running outdated hypervisor (qemu) binaries on this host.  
ubuntu@k8s-worker1:~$ containerd --version  
containerd github.com/containerd/containerd/v2 2.2.2  
ubuntu@k8s-worker1:~$
```

```
ubuntu@k8s-worker2: ~  
Unpacking runc (1.4.0-0ubuntu1) ...  
Selecting previously unselected package containerd.  
Preparing to unpack .../containerd_2.2.2-0ubuntu1.1_amd64v3.deb ...  
Unpacking containerd (2.2.2-0ubuntu1.1) ...  
Setting up runc (1.4.0-0ubuntu1) ...  
Setting up containerd (2.2.2-0ubuntu1.1) ...  
Created symlink '/etc/systemd/system/multi-user.target.wants/containerd.service'  
→ '/usr/lib/systemd/system/containerd.service'.  
Processing triggers for man-db (2.13.1-1build1) ...  
Scanning processes...  
Scanning linux images...  
  
Running kernel seems to be up-to-date.  
  
No services need to be restarted.  
  
No containers need to be restarted.  
  
No user sessions are running outdated binaries.  
  
No VM guests are running outdated hypervisor (qemu) binaries on this host.  
ubuntu@k8s-worker2:~$ containerd --version  
containerd github.com/containerd/containerd/v2 2.2.2  
ubuntu@k8s-worker2:~$ s|
```

13. Configure containerd

Create the configuration directory:

```
sudo mkdir -p /etc/containerd
```

Generate configuration:

```
sudo containerd config default | sudo tee /etc/containerd/config.toml
```

Open configuration:

```
sudo nano /etc/containerd/config.toml
```

Find:

```
SystemdCgroup = false
```

Change it to:

```
SystemdCgroup = true
```

Save:

CTRL + O

ENTER

CTRL + X

Restart containerd:

```
sudo systemctl restart containerd
```

Enable at boot:

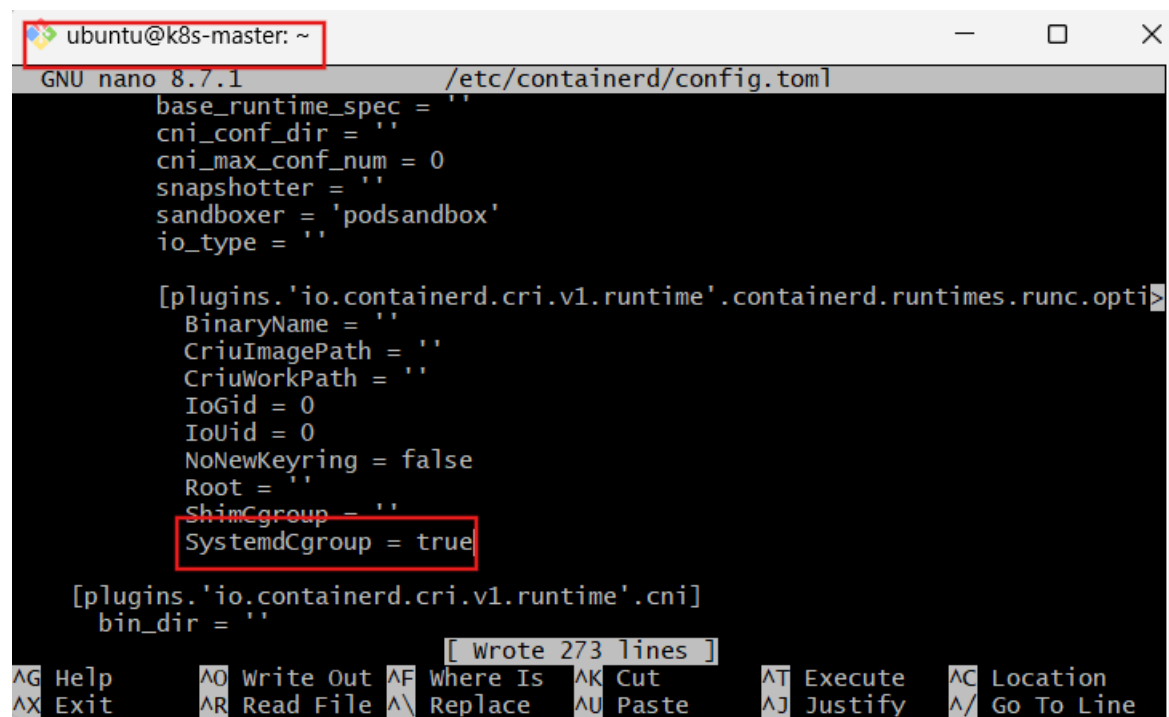
```
sudo systemctl enable containerd
```

Check:

```
sudo systemctl status containerd
```

You should see:

Active: active (running)



```
ubuntu@k8s-master: ~  
GNU nano 8.7.1 /etc/containerd/config.toml  
base_runtime_spec = ''  
cni_conf_dir = ''  
cni_max_conf_num = 0  
snapshotter = ''  
sandboxer = 'podsandbox'  
io_type = ''  
  
[plugins.'io.containerd.cri.v1.runtime'.containerd.runtimes.runc.opti  
  BinaryName = ''  
  CriuImagePath = ''  
  CriuWorkPath = ''  
  IoGid = 0  
  IoUid = 0  
  NoNewKeyring = false  
  Root = ''  
  ShimCgroup = ''  
  SystemdCgroup = true  
  
[plugins.'io.containerd.cri.v1.runtime'.cni]  
  bin_dir = ''  
  
[ Wrote 273 lines ]  
^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location  
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line
```

```
ubuntu@k8s-master: ~$ sudo systemctl status containerd
● containerd.service - containerd container runtime
   Loaded: loaded (/usr/lib/systemd/system/containerd.service; enabled; prese
   Active: active (running) since Tue 2026-09-22 09:43:31 UTC; 13s ago
   Invocation: 5f0d91670dd24afeb30d081eebc6718f
     Docs: https://containerd.io
   Main PID: 28895 (containerd)
     Tasks: 7
    Memory: 13.9M (peak: 16.8M)
       CPU: 69ms
    CGroup: /system.slice/containerd.service
            └─28895 /usr/bin/containerd

Sep 22 09:43:31 k8s-master containerd[28895]: time="2026-09-22T09:43:31.2794758>
Sep 22 09:43:31 k8s-master containerd[28895]: time="2026-09-22T09:43:31.2794814>
Sep 22 09:43:31 k8s-master containerd[28895]: time="2026-09-22T09:43:31.2794913>
Sep 22 09:43:31 k8s-master containerd[28895]: time="2026-09-22T09:43:31.2794949>
Sep 22 09:43:31 k8s-master containerd[28895]: time="2026-09-22T09:43:31.2794995>
Sep 22 09:43:31 k8s-master containerd[28895]: time="2026-09-22T09:43:31.2795204>
Sep 22 09:43:31 k8s-master containerd[28895]: time="2026-09-22T09:43:31.2795283>
Sep 22 09:43:31 k8s-master containerd[28895]: time="2026-09-22T09:43:31.2795374>
Sep 22 09:43:31 k8s-master containerd[28895]: time="2026-09-22T09:43:31.2796097>
Sep 22 09:43:31 k8s-master systemd[1]: Started containerd.service - containerd
ubuntu@k8s-master:~$
```

```
ubuntu@k8s-worker1: ~$ nano /etc/containerd/config.toml
GNU nano 8.7.1 /etc/containerd/config.toml
    NoNewKeyring = false
    Root = ''
    ShimCgroup = ''
    SystemdCgroup = true
[plugins.'io.containerd.cri.v1.runtime'.cni]
  bin_dir = ''
  bin_dirs = ['/opt/cni/bin']
  conf_dir = '/etc/cni/net.d'
  max_conf_num = 1
  setup_serially = false
  conf_template = ''
  ip_pref = ''
  use_internal_loopback = false

[plugins.'io.containerd.differ.v1.erofs']
  mkfs_options = []
  enable_tar_index = false

[plugins.'io.containerd.gc.v1.scheduler']
  pause_threshold = 0.02

[ Wrote 272 lines ]
^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute  ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify  ^_ Go To Line
```

```
ubuntu@k8s-worker1: ~  
ubuntu@k8s-worker1:~$ sudo systemctl status containerd  
● containerd.service - containerd container runtime  
   Loaded: loaded (/usr/lib/systemd/system/containerd.service; enabled; prese  
   Active: active (running) since Tue 2026-09-22 09:51:10 UTC; 22s ago  
   Invocation: f3bd63ce628645a4ae5f637a15e0f8eb  
     Docs: https://containerd.io  
   Main PID: 2447 (containerd)  
     Tasks: 7  
    Memory: 14.1M (peak: 16.9M)  
       CPU: 78ms  
    CGroup: /system.slice/containerd.service  
            └─2447 /usr/bin/containerd  
  
Sep 22 09:51:10 k8s-worker1 containerd[2447]: time="2026-09-22T09:51:10.1868639>  
Sep 22 09:51:10 k8s-worker1 containerd[2447]: time="2026-09-22T09:51:10.1869338>  
Sep 22 09:51:10 k8s-worker1 containerd[2447]: time="2026-09-22T09:51:10.1869432>  
Sep 22 09:51:10 k8s-worker1 containerd[2447]: time="2026-09-22T09:51:10.1869489>  
Sep 22 09:51:10 k8s-worker1 containerd[2447]: time="2026-09-22T09:51:10.1869564>  
Sep 22 09:51:10 k8s-worker1 containerd[2447]: time="2026-09-22T09:51:10.1869625>  
Sep 22 09:51:10 k8s-worker1 containerd[2447]: time="2026-09-22T09:51:10.1869672>  
Sep 22 09:51:10 k8s-worker1 containerd[2447]: time="2026-09-22T09:51:10.1869741>  
Sep 22 09:51:10 k8s-worker1 containerd[2447]: time="2026-09-22T09:51:10.1870723>  
Sep 22 09:51:10 k8s-worker1 systemd[1]: Started containerd.service - containerd  
ubuntu@k8s-worker1:~$
```

```
ubuntu@k8s-worker2: ~  
GNU nano 8.7.1 /etc/containerd/config.toml  
    snapshotter = ''  
    sandboxer = 'podsandbox'  
    io_type = ''  
  
    [plugins.'io.containerd.cri.v1.runtime'.containerd.runtimes.runc.opti  
      BinaryName = ''  
      CriuImagePath = ''  
      CriuWorkPath = ''  
      IoGid = 0  
      IoUid = 0  
      NoNewKeyring = false  
      Root = ''  
      ShimCgroup = ''  
      SystemdCgroup = true  
  
    [plugins.'io.containerd.cri.v1.runtime'.cni]  
      bin_dir = ''  
      bin_dirs = ['/opt/cni/bin']  
      conf_dir = '/etc/cni/net.d'  
      max_conf_num = 1
```

```
ubuntu@k8s-worker2: ~$ sudo systemctl status containerd
● containerd.service - containerd container runtime
   Loaded: loaded (/usr/lib/systemd/system/containerd.service; enabled; prese
   Active: active (running) since Tue 2026-09-22 09:54:44 UTC; 13s ago
   Invocation: 92f502e70fb442dab1748405c68e67f5
   Docs: https://containerd.io
   Main PID: 2508 (containerd)
   Tasks: 7
   Memory: 14.2M (peak: 16.9M)
   CPU: 63ms
   CGroup: /system.slice/containerd.service
           └─2508 /usr/bin/containerd

Sep 22 09:54:44 k8s-worker2 containerd[2508]: time="2026-09-22T09:54:44.6330571>
Sep 22 09:54:44 k8s-worker2 containerd[2508]: time="2026-09-22T09:54:44.6331083>
Sep 22 09:54:44 k8s-worker2 containerd[2508]: time="2026-09-22T09:54:44.6331731>
Sep 22 09:54:44 k8s-worker2 containerd[2508]: time="2026-09-22T09:54:44.6332144>
Sep 22 09:54:44 k8s-worker2 containerd[2508]: time="2026-09-22T09:54:44.6332624>
Sep 22 09:54:44 k8s-worker2 containerd[2508]: time="2026-09-22T09:54:44.6332137>
Sep 22 09:54:44 k8s-worker2 containerd[2508]: time="2026-09-22T09:54:44.6332836>
Sep 22 09:54:44 k8s-worker2 containerd[2508]: time="2026-09-22T09:54:44.6332930>
Sep 22 09:54:44 k8s-worker2 systemd[1]: Started containerd.service - containerd
Sep 22 09:54:44 k8s-worker2 containerd[2508]: time="2026-09-22T09:54:44.6336294>
ubuntu@k8s-worker2:~$
```

14. Install Kubernetes Components

We need three main Kubernetes packages:

kubeadm

kubelet

kubectrl

Run on **all three nodes**.

First install required packages:

sudo apt-get update

sudo apt-get install -y apt-transport-https ca-certificates curl gpg

Create keyring directory:

sudo mkdir -p -m 755 /etc/apt/keyrings

Add the Kubernetes signing key:

curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.34/deb/Release.key | \

sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg

Add repository:

```
echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg]
```

```
https://pkgs.k8s.io/core:/stable:/v1.34/deb/ ' | \
```

```
sudo tee /etc/apt/sources.list.d/kubernetes.list
```

Update:

```
sudo apt-get update
```

Install:

```
sudo apt-get install -y kubelet kubeadm kubectl
```

Prevent automatic version changes:

```
sudo apt-mark hold kubelet kubeadm kubectl
```

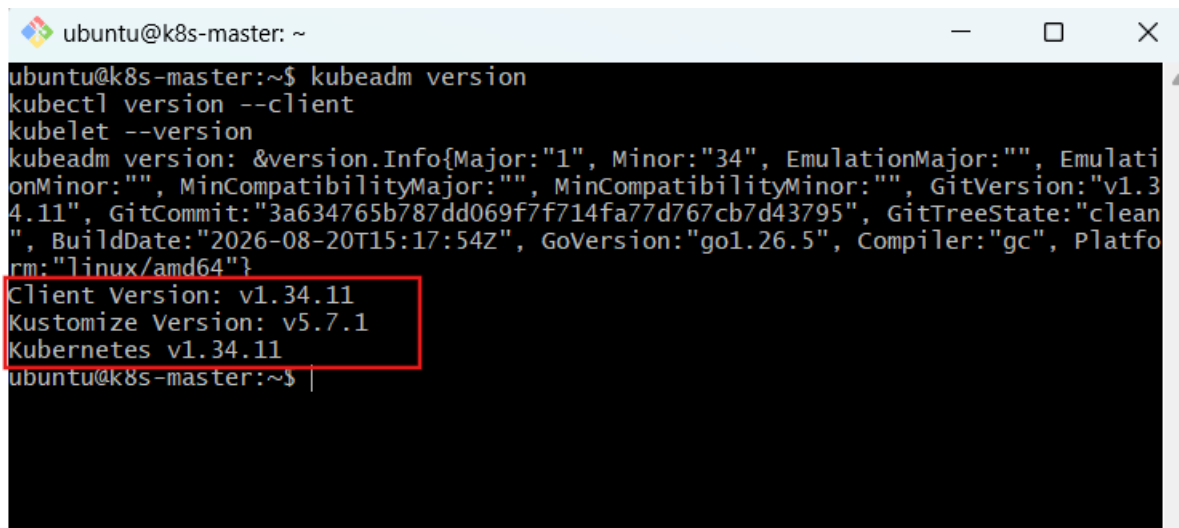
Check versions:

```
kubeadm version
```

```
kubectl version --client
```

```
kubelet --version
```

Keep the Kubernetes version consistent across all three nodes.

A terminal window titled 'ubuntu@k8s-master: ~' with standard window controls. It displays the output of three commands: 'kubeadm version', 'kubectl version --client', and 'kubelet --version'. The output for 'kubeadm version' is a long JSON string. The output for 'kubectl version --client' shows 'Client Version: v1.34.11'. The output for 'kubelet --version' shows 'Kubernetes v1.34.11'. A red rectangular box highlights the last two lines of output.

```
ubuntu@k8s-master:~$ kubeadm version
kubectl version --client
kubelet --version
kubeadm version: &version.Info{Major:"1", Minor:"34", EmulationMajor:"", EmulationMinor:"", MinCompatibilityMajor:"", MinCompatibilityMinor:"", GitVersion:"v1.34.11", GitCommit:"3a634765b787dd069f7f714fa77d767cb7d43795", GitTreeState:"clean", BuildDate:"2026-08-20T15:17:54Z", GoVersion:"go1.26.5", Compiler:"gc", Platform:"linux/amd64"}
Client Version: v1.34.11
Kustomize Version: v5.7.1
Kubernetes v1.34.11
ubuntu@k8s-master:~$
```

```
ubuntu@k8s-worker1: ~$ kubectl version --client
kubectl version --client
kubeadm version: &version.Info{Major:"1", Minor:"34", EmulationMajor:"", EmulationMinor:"", MinCompatibilityMajor:"", MinCompatibilityMinor:"", GitVersion:"v1.34.11", GitCommit:"3a634765b787dd069f7f714fa77d767cb7d43795", GitTreeState:"clean", BuildDate:"2026-08-20T15:17:54Z", GoVersion:"go1.26.5", Compiler:"gc", Platform:"linux/amd64"}
Client Version: v1.34.11
Kustomize Version: v5.7.1
Kubernetes v1.34.11
ubuntu@k8s-worker1:~$
```

```
ubuntu@k8s-worker2: ~$ kubectl version --client
kubectl version --client
kubeadm version: &version.Info{Major:"1", Minor:"34", EmulationMajor:"", EmulationMinor:"", MinCompatibilityMajor:"", MinCompatibilityMinor:"", GitVersion:"v1.34.11", GitCommit:"3a634765b787dd069f7f714fa77d767cb7d43795", GitTreeState:"clean", BuildDate:"2026-08-20T15:17:54Z", GoVersion:"go1.26.5", Compiler:"gc", Platform:"linux/amd64"}
Client Version: v1.34.11
Kustomize Version: v5.7.1
Kubernetes v1.34.11
ubuntu@k8s-worker2:~$
```

15. Initialize Kubernetes Master

Now switch to the **master node only**.

Run:

```
sudo kubeadm init --apiserver-advertise-address=<MASTER_PRIVATE_IP> --pod-network-cidr=192.168.0.0/16
```

Example:

```
sudo kubeadm init \
--apiserver-advertise-address=172.31.20.10 \
--pod-network-cidr=192.168.0.0/16
```

This process can take several minutes.

At the end, you should see something similar to:

Your Kubernetes control-plane has initialized successfully!

You will also receive a command similar to:

```
kubeadm join 172.31.20.10:6443 \
```

```
--token xxxxxxxxxxxxxxxxx \
```

```
--discovery-token-ca-cert-hash sha256:xxxxxxxxxxxxxxxxxx
```

VERY IMPORTANT

Save this kubeadm join command.

You will use it on both worker nodes.

```

ubuntu@k8s-master: ~
7/healthz
[control-plane-check] Checking kube-scheduler at https://127.0.0.1:10259/livez
[control-plane-check] kube-controller-manager is healthy after 8.4777ms
[control-plane-check] kube-scheduler is healthy after 11.340014ms
[control-plane-check] kube-apiserver is healthy after 1.501113144s
[upload-config] Storing the configuration used in ConfigMap "kubeadm-config" in
the "kube-system" Namespace
[kubelet] Creating a ConfigMap "kubelet-config" in namespace kube-system with th
e configuration for the kubelets in the cluster
[upload-certs] Skipping phase. Please see --upload-certs
[mark-control-plane] Marking the node k8s-master as control-plane by adding the
labels: [node-role.kubernetes.io/control-plane node.kubernetes.io/exclude-from-e
xternal-load-balancers]
[mark-control-plane] Marking the node k8s-master as control-plane by adding the
taints [node-role.kubernetes.io/control-plane:NoSchedule]
[bootstrap-token] Using token: 5cwpvx.yxut30zdx39gupu6
[bootstrap-token] Configuring bootstrap tokens, cluster-info ConfigMap, RBAC Rol
es
[bootstrap-token] Configured RBAC rules to allow Node Bootstrap tokens to get no
des
[bootstrap-token] Configured RBAC rules to allow Node Bootstrap tokens to post C
SRs in order for nodes to get long term certificate credentials
[bootstrap-token] Configured RBAC rules to allow the csrapprover controller auto
matically approve CSRs from a Node Bootstrap Token
[bootstrap-token] Configured RBAC rules to allow certificate rotation for all no
de client certificates in the cluster
[bootstrap-token] Configured RBAC rules to allow the API server kubelet client c
ertificate to access the kubelet API
[bootstrap-token] Creating the "cluster-info" ConfigMap in the "kube-public" nam
espace
[kubelet-finalize] Updating "/etc/kubernetes/kubelet.conf" to point to a rotatab
le kubelet client certificate and key
[addons] Applied essential addon: CoreDNS
[addons] Applied essential addon: kube-proxy

Your Kubernetes control-plane has initialized successfully!

To start using your cluster, you need to run the following as a regular user:

    mkdir -p $HOME/.kube
    sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
    sudo chown $(id -u):$(id -g) $HOME/.kube/config

Alternatively, if you are the root user, you can run:

    export KUBECONFIG=/etc/kubernetes/admin.conf

You should now deploy a pod network to the cluster.
Run "kubectl apply -f [podnetwork].yaml" with one of the options listed at:
    https://kubernetes.io/docs/concepts/cluster-administration/addons/

Then you can join any number of worker nodes by running the following on each as
root:

kubeadm join 172.31.17.127:6443 --token 5cwpvx.yxut30zdx39gupu6 \
--discovery-token-ca-cert-hash sha256:881aeaa85f14a1a913b87a9153f2de35d1
db73a3436af6e1275d2b98a0d5052e
ubuntu@k8s-master:~$

```

16. Configure kubectl on Master

Still on the master:

```
mkdir -p $HOME/.kube
```

Copy the Kubernetes configuration:

```
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
```

Change ownership:

```
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

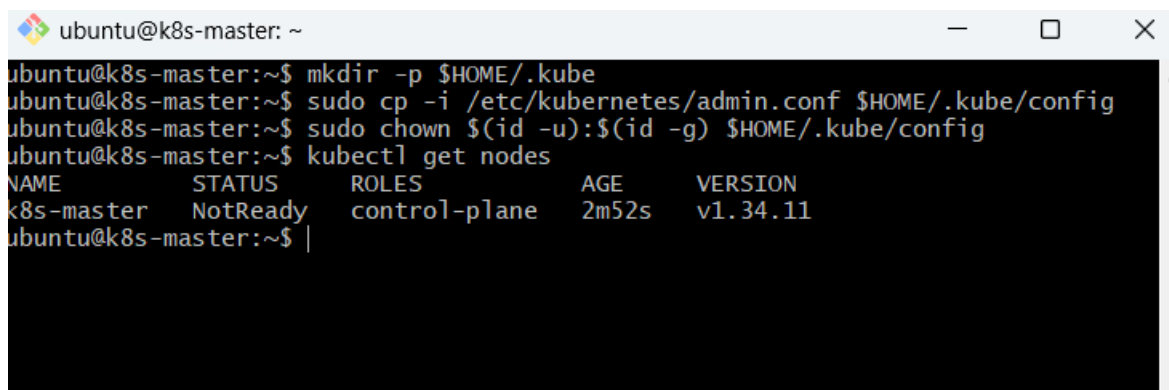
Test:

```
kubectl get nodes
```

Initially you should see:

NAME	STATUS	ROLES	AGE	VERSION
k8s-master	NotReady	control-plane	...	...

NotReady is expected because we haven't installed the pod network yet.



```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ mkdir -p $HOME/.kube  
ubuntu@k8s-master:~$ sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config  
ubuntu@k8s-master:~$ sudo chown $(id -u):$(id -g) $HOME/.kube/config  
ubuntu@k8s-master:~$ kubectl get nodes  
NAME          STATUS    ROLES    AGE   VERSION  
k8s-master    NotReady  control-plane  2m52s  v1.34.11  
ubuntu@k8s-master:~$ |
```

17. Install Calico Network Plugin

We need a Container Network Interface (CNI).

For this lab, use **Calico**.

On the master:

```
kubectl apply -f
```

```
https://raw.githubusercontent.com/projectcalico/calico/master/manifests/calico.yaml
```

Check:

```
kubectl get pods -n kube-system
```

Wait until the Calico pods become:

Running

Check nodes:

```
kubectl get nodes
```

The master should eventually become:

k8s-master Ready control-plane

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ mkdir -p $HOME/.kube  
ubuntu@k8s-master:~$ sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config  
ubuntu@k8s-master:~$ sudo chown $(id -u):$(id -g) $HOME/.kube/config  
ubuntu@k8s-master:~$ kubectl get nodes  
NAME          STATUS    ROLES    AGE      VERSION  
k8s-master    NotReady  control-plane  2m52s    v1.34.11  
ubuntu@k8s-master:~$ kubectl apply -f https://raw.githubusercontent.com/projectcalico/calico/master/manifests/calico.yaml  
poddisruptionbudget.policy/calico-kube-controllers created  
serviceaccount/calico-kube-controllers created  
serviceaccount/calico-node created  
configmap/calico-cni-plugin created  
configmap/calico-config created  
customresourcedefinition.apiextensions.k8s.io/bgpconfigurations.crd.projectcalico.org created  
customresourcedefinition.apiextensions.k8s.io/bgpfilters.crd.projectcalico.org created  
customresourcedefinition.apiextensions.k8s.io/bgppeers.crd.projectcalico.org created  
customresourcedefinition.apiextensions.k8s.io/clusternetnetworkpolicies.policy.networking.k8s.io created  
clusterrole.rbac.authorization.k8s.io/calico-kube-controllers created  
clusterrole.rbac.authorization.k8s.io/calico-node created  
clusterrole.rbac.authorization.k8s.io/calico-cni-plugin created  
clusterrole.rbac.authorization.k8s.io/calico-tier-getter created  
clusterrolebinding.rbac.authorization.k8s.io/calico-kube-controllers created  
clusterrolebinding.rbac.authorization.k8s.io/calico-node created  
clusterrolebinding.rbac.authorization.k8s.io/calico-cni-plugin created  
clusterrolebinding.rbac.authorization.k8s.io/calico-tier-getter created  
daemonset.apps/calico-node created  
deployment.apps/calico-kube-controllers created  
ubuntu@k8s-master:~$ kubectl get pods -n kube-system  
NAME                                READY   STATUS    RESTARTS   AGE  
calico-kube-controllers-f4cf54b75-bhgjr  0/1     Pending   0           10s  
calico-node-7bqzp                      0/1     Init:2/3   0           10s  
coredns-66bc5c9577-s4rsp               0/1     Pending   0           4m26s  
coredns-66bc5c9577-sdmpm               0/1     Pending   0           4m26s  
etcd-k8s-master                        1/1     Running    0           4m32s  
kube-apiserver-k8s-master               1/1     Running    0           4m32s  
kube-controller-manager-k8s-master      1/1     Running    0           4m32s  
kube-proxy-mqsvs                       1/1     Running    0           4m26s  
kube-scheduler-k8s-master               1/1     Running    0           4m32s  
ubuntu@k8s-master:~$ kubectl get nodes  
NAME          STATUS    ROLES    AGE      VERSION  
k8s-master    Ready     control-plane  4m49s    v1.34.11  
ubuntu@k8s-master:~$
```

18. Join Worker Node 1

Connect to Worker 1:

```
ssh -i ~/Downloads/docker.pem ubuntu@<WORKER1_PUBLIC_IP>
```

Run the **join** command generated by **kubeadm init**.

Example:

```
sudo kubeadm join 172.31.20.10:6443 \
```

```
--token xxxxxxxxxxxxxxxxxxxx \
```

```
--discovery-token-ca-cert-hash sha256:xxxxxxxxxxxxxxxxxx
```

You should see:

This node has joined the cluster

```

ubuntu@k8s-worker1: ~$ sudo kubeadm join 172.31.17.127:6443 --token 5cwpvx.yxut30
zdx39gupu6 \
--discovery-token-ca-cert-hash sha256:881aeea85f14a1a913b87a9153f2de35d1db73a3
436af6e1275d2b98a0d5052e
[preflight] Running pre-flight checks
[preflight] Reading configuration from the "kubeadm-config" ConfigMap in namespa
ce "kube-system"...
[preflight] Use 'kubeadm init phase upload-config kubeadm --config your-config-f
ile' to re-upload it.
W0922 10:17:24.003771    9112 utils.go:69] The recommended value for "bindAddres
s" in "KubeProxyConfiguration" is: ::; the provided value is: 0.0.0.0
[kubelet-start] Writing kubelet configuration to file "/var/lib/kubelet/instance
-config.yaml"
[patches] Applied patch of type "application/strategic-merge-patch+json" to targ
et "kubeletconfiguration"
[kubelet-start] Writing kubelet configuration to file "/var/lib/kubelet/config.y
aml"
[kubelet-start] Writing kubelet environment file with flags to file "/var/lib/ku
belet/kubeadm-flags.env"
[kubelet-start] Starting the kubelet
[kubelet-check] Waiting for a healthy kubelet at http://127.0.0.1:10248/healthz.
This can take up to 4m0s
[kubelet-check] The kubelet is healthy after 501.835919ms
[kubelet-start] Waiting for the kubelet to perform the TLS Bootstrap
This node has joined the cluster:
* Certificate signing request was sent to apiservert and a response was received.
* The Kubelet was informed of the new secure connection details.

Run 'kubectl get nodes' on the control-plane to see this node join the cluster.

ubuntu@k8s-worker1:~$ |

```

19. Join Worker Node 2

Connect to Worker 2:

```
ssh -i ~/Downloads/docker.pem ubuntu@<WORKER2_PUBLIC_IP>
```

Run the same join command:

```

sudo kubeadm join 172.31.20.10:6443 \
--token xxxxxxxxxxxxxxxxx \
--discovery-token-ca-cert-hash sha256:xxxxxxxxxxxxxxxxxxx

```

You should see:

This node has joined the cluster

```
ubuntu@k8s-worker2: ~  
ile' to re-upload it.  
w0922 10:19:33.490360    9061 utils.go:69] The recommended value for "bindAddress"  
s" in "KubeProxyConfiguration" is: ::; the provided value is: 0.0.0.0  
[kubelet-start] Writing kubelet configuration to file "/var/lib/kubelet/instance-  
-config.yaml"  
[patches] Applied patch of type "application/strategic-merge-patch+json" to targ  
et "kubeletconfiguration"  
[kubelet-start] Writing kubelet configuration to file "/var/lib/kubelet/config.y  
aml"  
[kubelet-start] Writing kubelet environment file with flags to file "/var/lib/ku  
belet/kubeadm-flags.env"  
[kubelet-start] Starting the kubelet  
[kubelet-check] Waiting for a healthy kubelet at http://127.0.0.1:10248/healthz.  
This can take up to 4m0s  
[kubelet-check] The kubelet is healthy after 501.054956ms  
[kubelet-start] Waiting for the kubelet to perform the TLS Bootstrap  
  
This node has joined the cluster:  
* Certificate signing request was sent to apiserver and a response was received.  
* The Kubelet was informed of the new secure connection details.  
  
Run 'kubectl get nodes' on the control-plane to see this node join the cluster.  
ubuntu@k8s-worker2:~$ |
```

20. Verify Kubernetes Cluster

Go back to the **master**.

Run:

kubectl get nodes

Expected:

NAME	STATUS	ROLES	AGE	VERSION
k8s-master	Ready	control-plane	...	...
k8s-worker1	Ready	<none>	...	...
k8s-worker2	Ready	<none>	...	...

This confirms:

1 Control Plane

|

+----- Worker 1

|

+----- Worker 2

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl get nodes  
NAME             STATUS    ROLES    AGE   VERSION  
k8s-master       Ready     control-plane  12m   v1.34.11  
k8s-worker1      Ready     <none>    4m3s  v1.34.11  
k8s-worker2      Ready     <none>    113s  v1.34.11  
ubuntu@k8s-master:~$ |
```

21. Check All System Pods

Run:

`kubectl get pods -A`

You should see Kubernetes system components such as:

`coredns`

`etcd-k8s-master`

`kube-apiserver-k8s-master`

`kube-controller-manager-k8s-master`

`kube-proxy`

`kube-scheduler-k8s-master`

`calico-node`

Most/all should show:

Running

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl get nodes  
NAME             STATUS    ROLES    AGE   VERSION  
k8s-master       Ready     control-plane  14m   v1.34.11  
k8s-worker1      Ready     <none>    5m42s  v1.34.11  
k8s-worker2      Ready     <none>    3m32s  v1.34.11  
ubuntu@k8s-master:~$ kubectl get pods -A  
NAMESPACE   NAME                                     READY   STATUS    RESTARTS   AGE  
kube-system  calico-kube-controllers-f4cf54b75-bhgjr  0/1     Running   7 (51s ago)  10m  
kube-system  calico-node-7bqzp                        1/1     Running   0           10m  
kube-system  calico-node-bpw4p                        1/1     Running   0           3m37s  
kube-system  calico-node-hgltg                        1/1     Running   0           5m47s  
kube-system  coredns-66bc5c9577-s4rsp                 1/1     Running   0           14m  
kube-system  coredns-66bc5c9577-sdmpt                 1/1     Running   0           14m  
kube-system  etcd-k8s-master                          1/1     Running   0           14m  
kube-system  kube-apiserver-k8s-master                 1/1     Running   0           14m  
kube-system  kube-controller-manager-k8s-master        1/1     Running   0           14m  
kube-system  kube-proxy-cwmmc                         1/1     Running   0           5m47s  
kube-system  kube-proxy-h8fls                         1/1     Running   0           3m37s  
kube-system  kube-proxy-mqsvs                         1/1     Running   0           14m  
kube-system  kube-scheduler-k8s-master                 1/1     Running   0           14m  
ubuntu@k8s-master:~$ |
```

22. Check Cluster Information

Run:

```
kubectl cluster-info
```

Example:

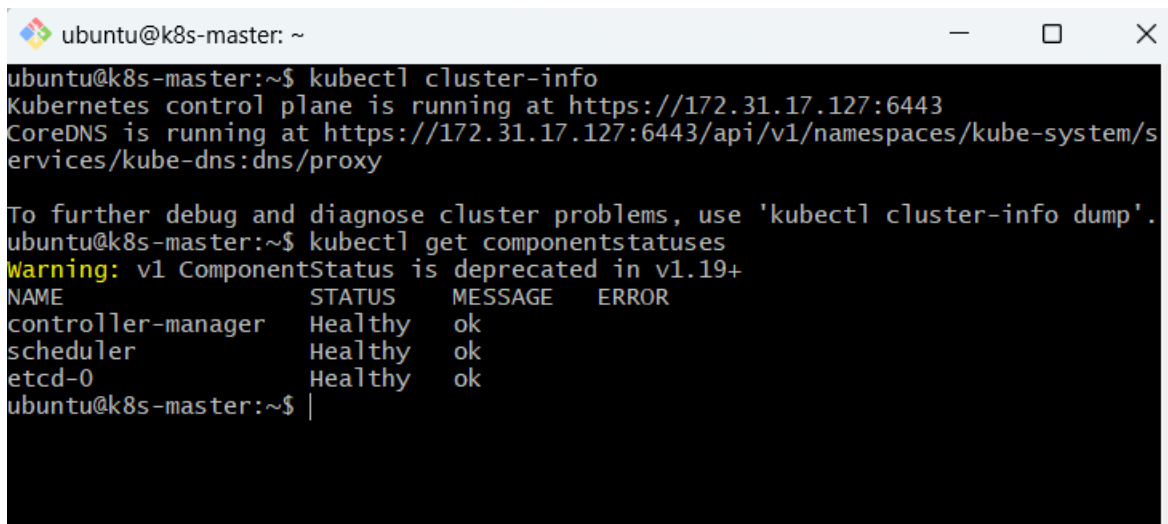
Kubernetes control plane is running at https://172.31.20.10:6443

You can also run:

```
kubectl get componentstatuses
```

Depending on your Kubernetes version, this command may be deprecated, so prefer:

```
kubectl get --raw='/readyz?verbose'
```



```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl cluster-info  
Kubernetes control plane is running at https://172.31.17.127:6443  
CoreDNS is running at https://172.31.17.127:6443/api/v1/namespaces/kube-system/s  
ervices/kube-dns:dns/proxy  
  
To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.  
ubuntu@k8s-master:~$ kubectl get componentstatuses  
Warning: v1 ComponentStatus is deprecated in v1.19+  
NAME                STATUS    MESSAGE    ERROR  
controller-manager  Healthy   ok           
scheduler            Healthy   ok           
etcd-0              Healthy   ok           
ubuntu@k8s-master:~$ |
```

3.Run one nginx pod

Run One Nginx Pod in Kubernetes

Since your Kubernetes cluster has **1 master + 2 worker nodes**, we can create one Nginx Pod and verify that it is running successfully.

Objective

Deploy **one Nginx Pod** in the Kubernetes cluster and verify:

- Pod creation
- Pod status

- Pod IP
 - Worker node where it is running
 - Nginx container
 - Nginx response
-

Step 1: Connect to the Kubernetes Master

From your Windows Git Bash:

```
ssh -i ~/Downloads/docker.pem ubuntu@<MASTER_PUBLIC_IP>
```

Example:

```
ssh -i ~/Downloads/docker.pem ubuntu@54.x.x.x
```

Verify Kubernetes:

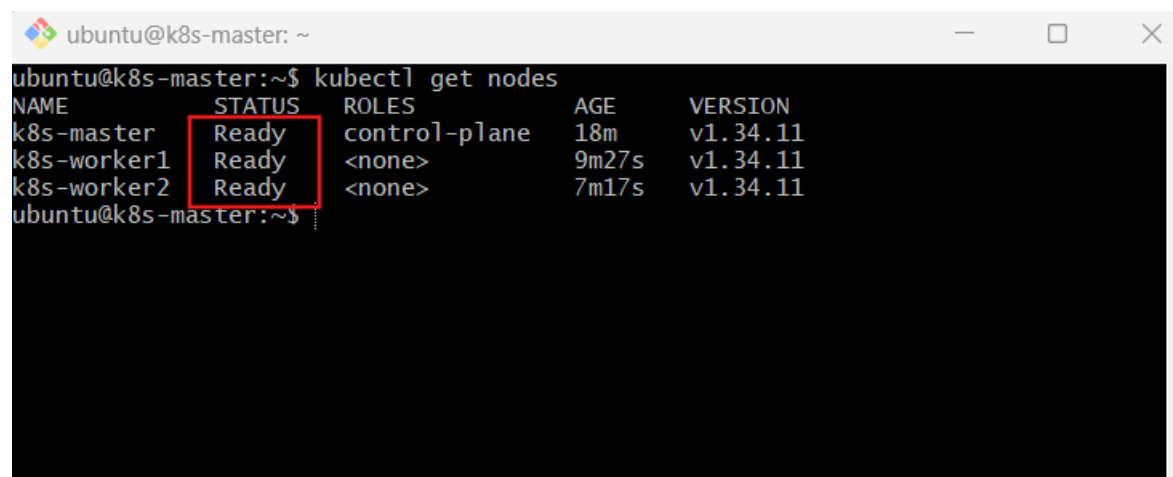
```
kubectl get nodes
```

You should see something similar to:

NAME	STATUS	ROLES	AGE
k8s-master	Ready	control-plane	...
k8s-worker1	Ready	<none>	...
k8s-worker2	Ready	<none>	...

All three nodes should ideally show:

Ready



A terminal window titled 'ubuntu@k8s-master: ~' showing the command 'kubectl get nodes' and its output. The output is a table with columns: NAME, STATUS, ROLES, AGE, and VERSION. The STATUS column for all three nodes (k8s-master, k8s-worker1, k8s-worker2) is 'Ready', which is highlighted with a red box. The terminal text is as follows:

```
ubuntu@k8s-master:~$ kubectl get nodes
NAME          STATUS    ROLES    AGE   VERSION
k8s-master    Ready     control-plane  18m   v1.34.11
k8s-worker1   Ready     <none>      9m27s v1.34.11
k8s-worker2   Ready     <none>      7m17s v1.34.11
ubuntu@k8s-master:~$
```

Step 2: Create an Nginx Pod

Run:

```
kubectl run nginx --image=nginx
```

Expected output:

```
pod/nginx created
```

This command means:

```
kubectl run
```

↓

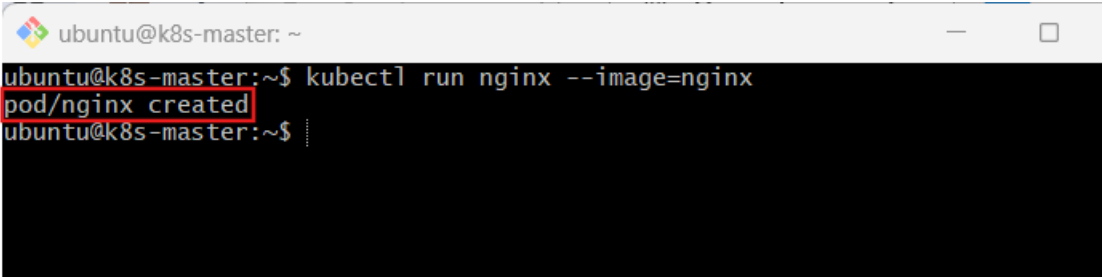
Create a Pod

↓

Pod name = nginx

↓

Container image = nginx

A terminal window titled 'ubuntu@k8s-master: ~' with standard window controls. The command 'kubectl run nginx --image=nginx' is entered at the prompt. The output 'pod/nginx created' is displayed on the next line and is highlighted with a red rectangular box. The prompt returns to 'ubuntu@k8s-master:~\$' on the following line.

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl run nginx --image=nginx  
pod/nginx created  
ubuntu@k8s-master:~$
```

Step 4: Get Detailed Pod Information

Run:

```
kubectl get pod nginx -o wide
```

Example:

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
nginx	1/1	Running	0	1m	192.168.1.5	k8s-worker1

This tells you:

- Pod name: nginx
- Status: Running

- Pod IP: 192.168.1.5
- Node: k8s-worker1

The Pod can run on either k8s-worker1 or k8s-worker2; Kubernetes scheduler decides where to place it.

```
ubuntu@k8s-master: ~$ kubectl run nginx --image=nginx
pod/nginx created
ubuntu@k8s-master: ~$ kubectl get pods
NAME      READY   STATUS    RESTARTS   AGE
nginx     1/1     Running   0           39s
ubuntu@k8s-master: ~$ kubectl get pod nginx -o wide
NAME      READY   STATUS    RESTARTS   AGE   IP            NODE       NOMINATED NODE
nginx     1/1     Running   0           73s   192.168.194.65 k8s-worker1 <none>
ubuntu@k8s-master: ~$ kubectl get pod nginx -o wide
NAME      READY   STATUS    RESTARTS   AGE   IP            NODE       NOMINATED NODE   READINESS GATES
nginx     1/1     Running   0           90s   192.168.194.65 k8s-worker1 <none>         <none>
```

Step 5: Describe the Pod

Run:

`kubectl describe pod nginx`

You will see information such as:

Name: nginx

Namespace: default

Node: k8s-worker1

Status: Running

IP: 192.168.x.x

Containers:

nginx:

Image: nginx

You can use this command when troubleshooting Pods.

```
ubuntu@k8s-master:~$ kubectl describe pod nginx
Name: nginx
Namespace: default
Priority: 0
Service Account: default
Node: k8s-worker1/172.31.16.155
Start Time: Tue, 22 Sep 2026 10:27:34 +0000
Labels: run=nginx
Annotations: cnf.projectcalico.org/containerID: 7b4d40d48fbc73af05bc07a9e6c740d29df5d7d716800f663e93f4101eddd3d4
  cnf.projectcalico.org/podIP: 192.168.194.65/32
  cnf.projectcalico.org/podIPs: 192.168.194.65/32
Status: Running
IP: 192.168.194.65
IPs: 192.168.194.65
Containers:
  nginx:
    Container ID: containerd://595ade1b5134314bb2dcdb4b5badd0409f2d897895520a2c823fa058c5e78ad6
    Image: nginx
    Image ID: docker.io/library/nginx@sha256:abe47724e466aeb9a345d8e46a221c2fa8953c7848bb4a3bd9976a7199f8cf2
    Port: <none>
    Host Port: <none>
    State: Running
      Started: Tue, 22 Sep 2026 10:27:57 +0000
    Ready: True
    Restart Count: 0
    Environment: <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-gz8qm (ro)
Conditions:
  Type              Status
  PodReadyToStartContainers  True
  Initialized         True
  Ready               True
  ContainersReady      True
  PodScheduled         True
Volumes:
  kube-api-access-gz8qm:
    Type: Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds: 3607
    ConfigMapName: kube-root-ca.crt
    Optional: false
    DownwardAPI: true
QoS Class: BestEffort
Node-Selectors: <none>
Tolerations: node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
              node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Events:
  Type    Reason      Age    From          Message
  ----    -
  Normal  Scheduled   4m4s   default-scheduler  Successfully assigned default/nginx to k8s-worker1
  Normal  Pulling     4m4s   kubelet         Pulling image "nginx"
  Normal  Pulled      4m2s   kubelet         Successfully pulled image "nginx" in 2.261s (2.261s including waiting). Image size: 63422407 bytes.
  Normal  Created     4m2s   kubelet         Created container: nginx
  Normal  Started     4m1s   kubelet         Started container nginx
ubuntu@k8s-master:~$
```

Step 7: Check the Nginx Version

Run:

```
kubectl exec nginx -- nginx -v
```

Example:

```
nginx version: nginx/1.x.x
```

This confirms that Nginx is actually running inside the container.

Step 6: Check Nginx Container Logs

Run:

```
kubectl logs nginx
```

You should see Nginx startup/access information.

For example:

/docker-entrypoint.sh: ...

Starting nginx

Step 7: Check the Nginx Version

Run:

```
kubectl exec nginx -- nginx -v
```

Example:

nginx version: nginx/1.x.x

This confirms that Nginx is actually running inside the container.

Step 8: Test Nginx Inside the Pod

Run:

```
kubectl exec nginx -- curl -I http://localhost
```

Expected response:

HTTP/1.1 200 OK

Server: nginx

If curl is not available inside the Nginx image, use:

```
kubectl exec nginx -- nginx -t
```

Expected:

syntax is ok

test is successful

Step 9: View the Nginx HTML Page

You can also retrieve the default Nginx page:

```
kubectl exec nginx -- cat /usr/share/nginx/html/index.html
```

You should see HTML content containing:

Welcome to nginx!

Step 10: Check Pod YAML

Run:

```
kubectl get pod nginx -o yaml
```

This displays the complete Kubernetes configuration and current state of the Pod.

For a shorter view:

```
kubectl get pod nginx -o json
```

Step 11: View Pod Events

Run:

```
kubectl describe pod nginx
```

At the bottom, look for:

Events:

You may see events such as:

Scheduled

Pulling

Pulled

Created

Started

This is useful for understanding how Kubernetes created the Pod.

Step 12: Test the Pod Using Port Forwarding

A Pod's IP is normally accessible only inside the Kubernetes network.

For an easy test from the **master machine**, use:

```
kubectl port-forward pod/nginx 8080:80
```

You should see:

Forwarding from 127.0.0.1:8080 -> 80

Keep this terminal running.

Open another SSH session to the master and execute:

```
curl http://127.0.0.1:8080
```

You should receive the Nginx HTML page.

You can also use:

```
curl -I http://127.0.0.1:8080
```

Expected:

HTTP/1.1 200 OK

Press:

CTRL + C

to stop port forwarding.

Step 13: Verify the Pod Again

Run:

```
kubectl get pods -o wide
```

Expected:

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
nginx	1/1	Running	0	5m	192.168.x.x	k8s-worker1

Step 14: Check Which Worker Is Running Nginx

Run:

```
kubectl get pod nginx -o wide
```

Look at the:

NODE

column.

For example:

NODE

k8s-worker1

Therefore:

Kubernetes Master

|

|

Kubernetes Scheduler

|

↓

k8s-worker1

|

↓

Nginx Pod

|

↓

Nginx Container

Step 15: Delete the Nginx Pod

When your testing is complete:

```
kubectl delete pod nginx
```

Expected:

```
pod "nginx" deleted
```

Verify:

```
kubectl get pods
```

You should see:

No resources found in default namespace.

Complete Command List

For your documentation, you can keep this short command sequence:

1. Check cluster

```
kubectl get nodes
```

2. Create Nginx Pod

```
kubectl run nginx --image=nginx
```

3. Check Pod

```
kubectl get pods
```

4. Check Pod with node/IP

```
kubectl get pod nginx -o wide
```

5. Describe Pod

```
kubectl describe pod nginx
```

6. Check logs

```
kubectl logs nginx
```

7. Check Nginx

```
kubectl exec nginx -- nginx -v
```

8. Test Nginx

```
kubectl exec nginx -- nginx -t
```

9. Port forward

```
kubectl port-forward pod/nginx 8080:80
```

10. Test from another terminal

```
curl http://127.0.0.1:8080
```

11. Delete Pod

```
kubectl delete pod nginx
```

Expected Final Result

Kubernetes Cluster

```
|
+--- Master
|
+--- Worker 1
|   |
|   +--- Nginx Pod
|
+--- Worker 2
```

The key verification command for your document is:

```
kubectl get pod nginx -o wide
```

with:

```
READY STATUS
```

```
1/1 Running
```

This confirms that **one Nginx Pod is successfully running in your Kubernetes cluster.**

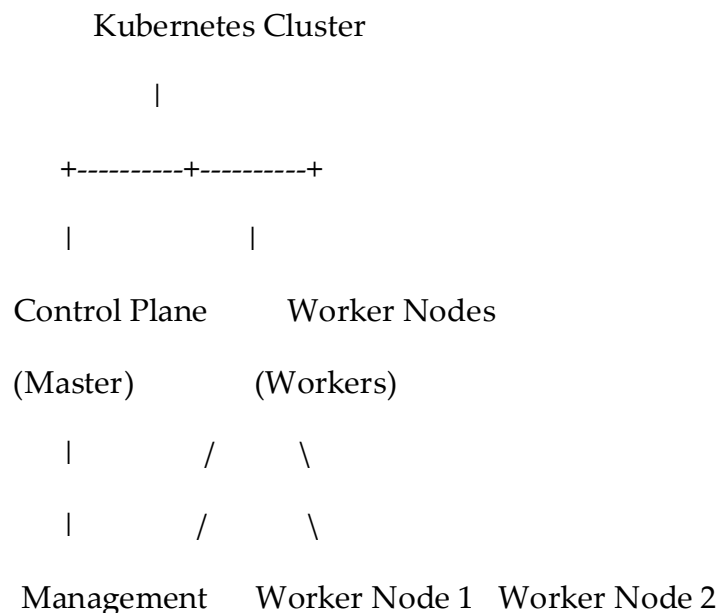
4. Master and slave components on k8s.

Kubernetes Master and Worker Components

Note: In modern Kubernetes terminology, “**Master**” is called the “**Control Plane**”, and “**Slave**” is called the “**Worker Node**.” In your documentation, use **Control Plane + Worker Node**.

1. Kubernetes Architecture

A Kubernetes cluster mainly has two types of nodes:



The simplest way to understand it:

Control Plane

Decides what should happen.

Worker Node

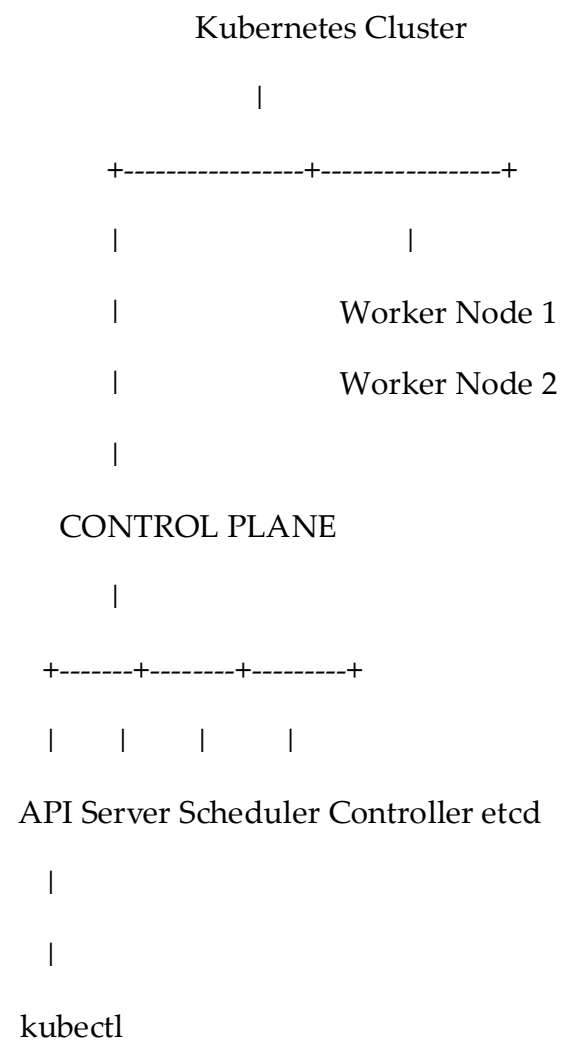
Actually runs the applications.

For example, when you execute:

```
kubectl create deployment nginx --image=nginx
```

the Control Plane receives the request, decides where the Pod should run, and the Worker Node actually starts the Nginx container.

2. Complete Kubernetes Architecture



Let's understand every component one by one.

3. Control Plane Components

The Control Plane contains the main Kubernetes management components:

Control Plane

|

├─ kube-apiserver

├─ etcd

|— kube-scheduler

|— kube-controller-manager

|— cloud-controller-manager

Not every cluster uses cloud-controller-manager; it is commonly present when Kubernetes integrates with a cloud provider.

4. kube-apiserver

What is kube-apiserver?

kube-apiserver is the **front door of Kubernetes**.

All requests to Kubernetes normally go through the API Server.

For example:

kubectl get pods

Your request travels approximately like this:

You

|

kubectl

|

v

kube-apiserver

|

+----> etcd

|

+----> Scheduler

|

+----> Controllers

Important point

The API Server is responsible for:

- Receiving Kubernetes API requests
 - Authentication
 - Authorization
 - Admission control
 - Validating requests
 - Communicating with other Kubernetes components
 - Providing the Kubernetes API
-

5. Example: `kubectl get nodes`

When you execute:

`kubectl get nodes`

the process is approximately:

`kubectl`

|

| HTTPS request

↓

`kube-apiserver`

|

↓

`etcd`

|

↓

`kube-apiserver`

|

↓

kubectl

|

↓

Display nodes

So remember:

kubectl talks to the API Server.

6. etcd

What is etcd?

etcd is the **database of Kubernetes**.

It stores the cluster's persistent state and configuration.

For example:

Cluster information

Node information

Pod information

Deployment information

Service information

Secrets

ConfigMaps

Conceptually:

Kubernetes

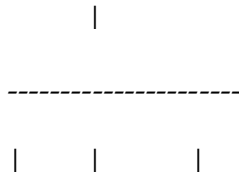
|

kube-apiserver

|

↓

etcd



Pods Nodes Services

Important point

etcd does **not run your applications**.

It stores Kubernetes state.

7. Example of etcd

Suppose you create:

```
kubectl create deployment nginx --image=nginx
```

Kubernetes needs to remember information such as:

Deployment name = nginx

Replicas = 1

Image = nginx

That state is stored through the Kubernetes API in etcd.

8. kube-scheduler

What does the Scheduler do?

The Scheduler decides:

Which Worker Node should run this Pod?

Suppose you have:

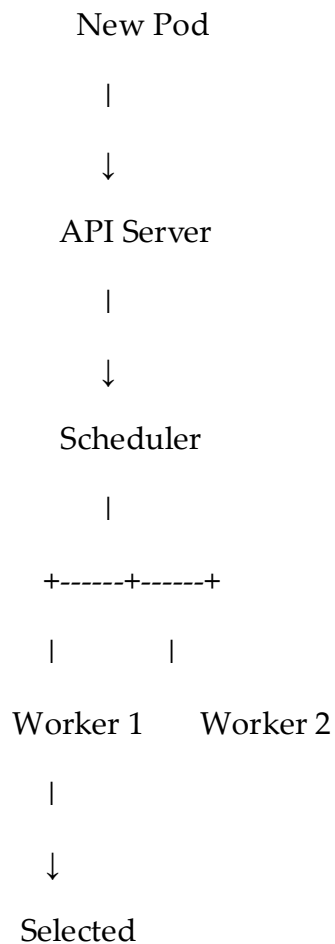
Worker 1

Worker 2

and create:

Nginx Pod

The Scheduler evaluates available nodes and selects an appropriate one.



Important point

The Scheduler **does not normally start the container itself**.

It selects a suitable node and records the scheduling decision.

9. How Scheduler Makes a Decision

The Scheduler considers factors such as:

- Available resources
- CPU
- Memory
- Node selectors
- Affinity/anti-affinity

- Taints and tolerations
- Pod requirements
- Other scheduling constraints

For example:

Pod requires:

CPU = 500m

Memory = 512 MB

The Scheduler checks available nodes.

Worker 1

CPU available: 2 CPU

Memory: 4 GB

↓

Suitable

Worker 2

CPU available: 100m

Memory: 100 MB

↓

Not suitable

It can therefore select Worker 1.

10. kube-controller-manager

This is one of the most important components.

The Controller Manager runs various controllers.

The basic idea is:

Controllers continuously compare the desired state with the current state and take action to move the cluster toward the desired state.

11. Desired State vs Current State

Suppose you create:

replicas: 3

You are saying:

Desired State = 3 Pods

Initially:

Current State = 0 Pods

The controller notices:

Desired = 3

Current = 0

So Kubernetes creates Pods.

Eventually:

Desired = 3

Current = 3

The controller continues monitoring.

12. Example: Pod Failure

Suppose you have:

Desired:

3 Pods

Current:

3 Pods

One Pod crashes:

Current:

2 Pods

The controller detects:

Desired = 3

Current = 2

It takes action to bring the cluster back toward:

3 Pods

This is one of the fundamental ideas behind Kubernetes' self-healing behavior.

13. Important Controllers

The Controller Manager can run controllers responsible for things such as:

Node Controller

Deployment/ReplicaSet related controllers

EndpointSlice-related controllers

Job Controller

ServiceAccount Controller

Namespace Controller

You don't normally manage each controller individually.

They are managed through the Kubernetes control plane.

14. cloud-controller-manager

In cloud environments such as AWS, Kubernetes can use:

cloud-controller-manager

It allows Kubernetes to interact with cloud-provider infrastructure.

For example:

Kubernetes

|

cloud-controller-manager

|

↓

AWS

|

EC2 / Load Balancers / Routes etc.

This component is not necessarily present in every Kubernetes installation.

15. Worker Node Components

Now let's move to the Worker Node.

A Worker Node primarily contains:

Worker Node

|

├── kubelet

├── kube-proxy

└── Container Runtime

The container runtime in your lab is:

containerd

16. kubelet

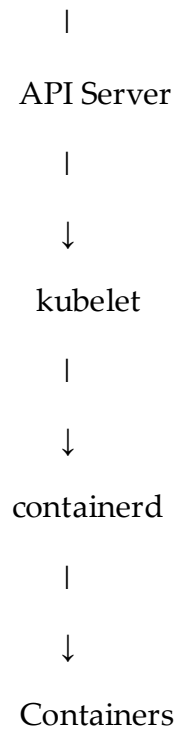
kubelet is the **main Kubernetes agent running on every Worker Node.**

Think of kubelet as:

The worker-node agent responsible for making sure the Pods assigned to that node are running.

Architecture:

Control Plane



17. What Does kubelet Do?

Kubelet:

- Communicates with the API Server
- Receives Pod specifications
- Ensures required containers are running
- Monitors containers
- Reports node and Pod status
- Performs health-related actions
- Works with the container runtime

18. Example of kubelet

Suppose the Scheduler assigns:

nginx Pod



Worker 1

The API Server records the assignment.

The kubelet on Worker 1 sees that the Pod should run there.

Then kubelet communicates with the container runtime:

API Server



kubelet



containerd



nginx container

19. Container Runtime

Kubernetes needs a container runtime to actually run containers.

In your setup:

containerd

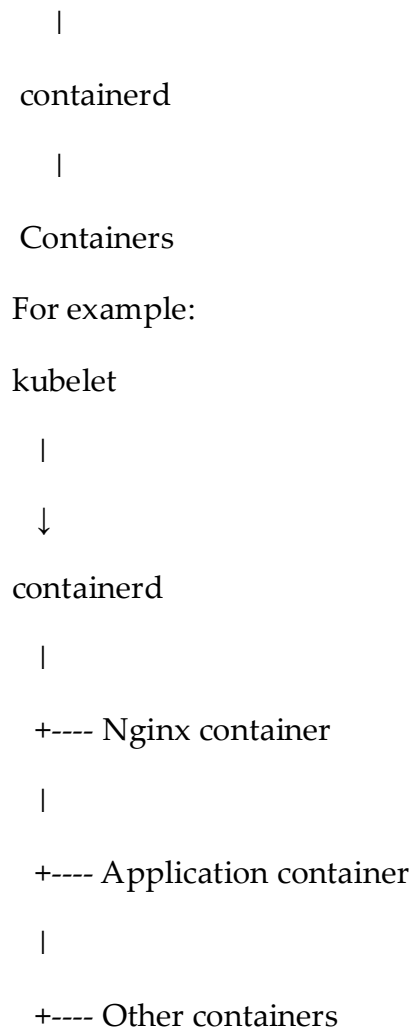
is the container runtime.

The relationship is:

Kubernetes



kubelet

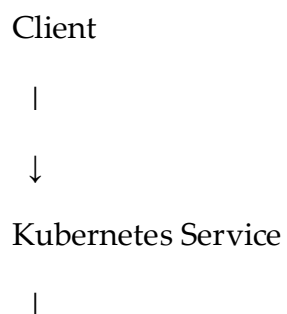


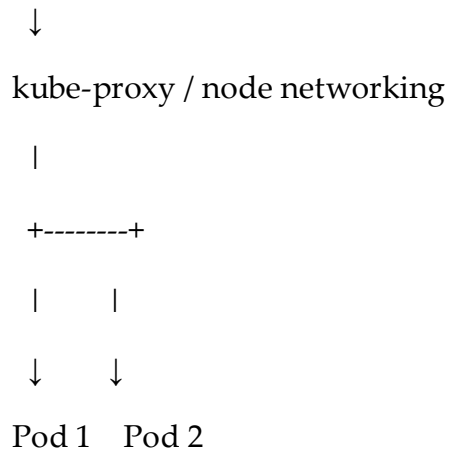
20. kube-proxy

kube-proxy runs on Worker Nodes and is associated with Kubernetes Service networking.

It helps implement network connectivity for Kubernetes Services using the node's networking facilities.

Conceptually:





For example:

Service:

nginx-service

Backend Pods:

nginx-pod-1

nginx-pod-2

nginx-pod-3

Traffic sent to the Service can be directed toward the backend Pods.

21. CNI Plugin

There is another important component in your cluster:

CNI

CNI means:

Container Network Interface

In your previous setup, we used:

Calico

Calico provides networking functionality for Kubernetes Pods.

Conceptually:

Pod

|

Calico

|

Kubernetes Network

|

Other Pods

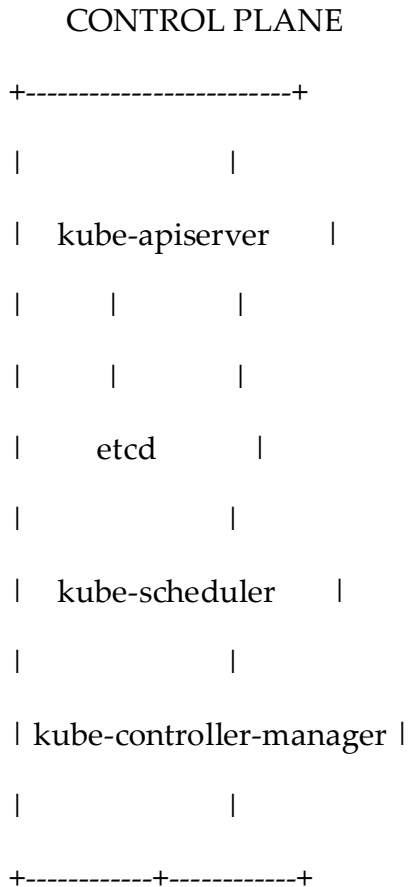
Without a correctly configured CNI, your nodes may remain:

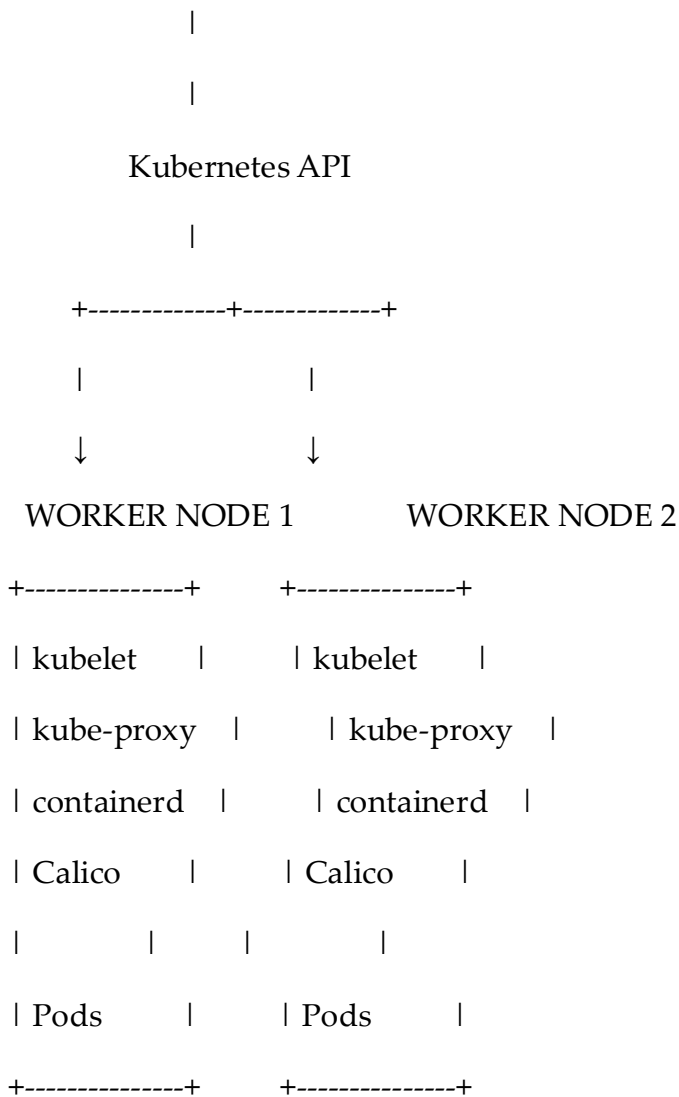
NotReady

and Pods may have networking problems.

22. Complete Component Architecture

Now combine everything:





23. Real-Time Example

Let's understand everything with your Nginx example.

You run:

```
kubectl run nginx --image=nginx
```

Step 1 — kubectl

kubectl sends the request to:

kube-apiserver

Step 2 — API Server

API Server validates and processes the request.

It stores the relevant cluster state through:

etcd

Step 3 — Scheduler

Scheduler notices a Pod needs a node.

It selects:

k8s-worker1

Step 4 — kubelet

The kubelet on Worker 1 notices that the Pod is assigned to it.

Step 5 — containerd

Kubelet asks the container runtime to create/run the container.

kubelet

↓

containerd

↓

Nginx container

Step 6 — CNI

Calico provides the Pod's network connectivity.

Step 7 — kube-proxy

If you expose the application through a Kubernetes Service, kube-proxy participates in implementing Service traffic routing on the node.

Step 8 — Status

Worker 1 reports Pod status through the API Server.

Eventually:

kubectl get pods

shows:

NAME	READY	STATUS	RESTARTS
------	-------	--------	----------

24. Easy Way to Remember Components

For interviews and classroom explanations, remember this:

Control Plane

API Server



"Talk to Kubernetes"

etcd



"Store Kubernetes data"

Scheduler



"Choose the Worker"

Controller Manager



"Maintain desired state"

Worker Node

kubelet



"Manage Pods on this node"

containerd



"Run containers"

kube-proxy



"Implement Service networking"

CNI / Calico



"Provide Pod networking"

25. One-Line Interview Answers

Component	Simple explanation
kube-apiserver	Entry point/API for the Kubernetes cluster
etcd	Stores Kubernetes cluster state
kube-scheduler	Selects a suitable node for Pods
kube-controller-manager	Maintains the desired state
cloud-controller-manager	Integrates Kubernetes with cloud infrastructure
kubelet	Agent that manages Pods on a node
containerd	Container runtime that runs containers
kube-proxy	Helps implement Kubernetes Service networking
CNI/Calico	Provides Pod networking

26. Most Important Interview Question

Q: What happens when you create a Pod?

A good answer is:

When I create a Pod using `kubectl`, the request goes to the Kubernetes API Server. The API Server validates the request and records the desired state. The Scheduler identifies a suitable Worker Node and assigns the Pod to it. The kubelet on that Worker Node observes the assignment and asks the container runtime, such as `containerd`, to start the required container. The CNI plugin provides Pod networking, and the kubelet continuously reports the Pod's status back to the API Server.

The flow is:

`kubectl`



API Server



`etcd`



Scheduler



Worker Node



kubelet



`containerd`



Container



Pod

27. Commands to See These Components on Your Cluster

On your **master/control-plane node**:

Control Plane Pods

```
kubectl get pods -n kube-system
```

You should see components such as:

```
etcd-k8s-master
```

```
kube-apiserver-k8s-master
```

```
kube-controller-manager-k8s-master
```

```
kube-scheduler-k8s-master
```

Worker-related Pods

```
kubectl get pods -n kube-system -o wide
```

Look for:

```
calico-node
```

```
kube-proxy
```

Check kubelet

On a worker:

```
sudo systemctl status kubelet
```

Check containerd

```
sudo systemctl status containerd
```

Check nodes

```
kubectl get nodes -o wide
```

28. Final Memory Trick

Think of Kubernetes like a **company**:

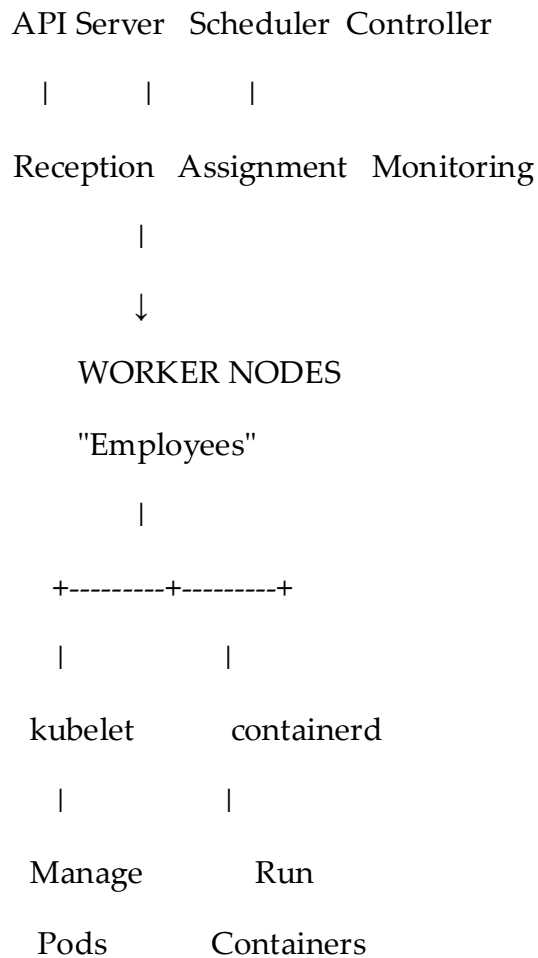
CONTROL PLANE

"Management Team"

|

+-----+-----+

| | |



The simplest formula to memorize is:

API Server → Communication

etcd → Database

Scheduler → Placement

Controller → Desired State

kubelet → Pod Management

containerd → Container Execution

kube-proxy → Service Networking

Calico → Pod Networking

This is the **core Kubernetes architecture** you should understand before moving on to Deployments, ReplicaSets, Services, ConfigMaps, Secrets, Ingress, and StatefulSets.

